



Module M16

Partha Pratim
Das

Weekly Recap

Objectives &
Outline

static Data
Member

Example

Print Task

Order of Initialization

static Member
function

Print Task

Count Objects

Comparison

Singleton Class

Module Summary

Programming in Modern C++

Module M16: static Members

Partha Pratim Das

Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur

ppd@cse.iitkgp.ac.in

All url's in this module have been accessed in September, 2021 and found to be functional



Weekly Recap

Module M16

Partha Pratim Das

Weekly Recap

Objectives & Outline

static Data Member

Example

Print Task

Order of Initialization

static Member function

Print Task

Count Objects

Comparison

Singleton Class

Module Summary

- Understood the core OOP features of C++: **Class**, **Object**, **Attributes** (Data members), **Methods** (Member functions), **State** of an Object, **Encapsulation** by Access Specifiers, get-set idioms for **Information Hiding**, various types of **Constructors**, **Destructors**, and **copy mechanisms** in terms of deep & shallow copy
- Understood the **lifetime** aspects of objects
- Understood how **bit-wise const-ness** can be modeled for objects and leveraged in the design with **const** objects, data members, and member functions, and how **mutable** can help model **logical const-ness**



Module Objectives

Module M16

Partha Pratim
Das

Weekly Recap

Objectives &
Outline

static Data
Member

Example

Print Task

Order of Initialization

static Member
function

Print Task

Count Objects

Comparison

Singleton Class

Module Summary

- Understand `static` data member and member function

NPTEL



Module Outline

Module M16

Partha Pratim Das

Weekly Recap

Objectives & Outline

static Data Member

Example

Print Task

Order of Initialization

static Member function

Print Task

Count Objects

Comparison

Singleton Class

Module Summary

1 Weekly Recap

2 static Data Member

- Example
- Print Task
- Order of Initialization

3 static Member function

- Print Task
- Count Objects

4 Comparison

5 Singleton Class

6 Module Summary



static Data Member

Module M16

Partha Pratim
Das

Weekly Recap

Objectives &
Outline

**static Data
Member**

Example

Print Task

Order of Initialization

static Member
function

Print Task

Count Objects

Comparison

Singleton Class

Module Summary

static Data Member



static Data Member

Module M16

Partha Pratim Das

Weekly Recap

Objectives & Outline

static Data Member

Example

Print Task

Order of Initialization

static Member function

Print Task

Count Objects

Comparison

Singleton Class

Module Summary

- A **static** data member
 - is *associated with class* not with object
 - is *shared by all the objects* of a class
 - needs to be *defined outside the class scope* (in addition to the *declaration within the class scope*) to avoid linker error
 - *must be initialized* in a source file
 - is constructed before `main()` starts and destructed after `main()` ends
 - can be **private** / **public**
 - can be accessed
 - ▷ with the class-name followed by the scope resolution operator (`::`)
 - ▷ as a member of any object of the class
 - **virtually eliminates any need for global variables in OOPs environment**
- We illustrate first with a simple example and then with a Print Task where:
 - There is a printer which can be loaded with a paper from time to time
 - Several print jobs (each requiring a number of pages) may be fired on the printer



Program 16.01: static Data Member: Example

Module M16

Partha Pratim Das

Weekly Recap

Objectives & Outline

static Data Member

Example

Print Task

Order of Initialization

static Member function

Print Task

Count Objects

Comparison

Singleton Class

Module Summary

Non static Data Member

```
#include<iostream>
using namespace std;
class MyClass { int x; // Non-static
public:
    void get() { x = 15; }
    void print() { x = x + 10;
        cout << "x =" << x << endl ;
    }
};

int main() {
    MyClass obj1, obj2; // Have distinct x
    obj1.get(); obj2.get();
    obj1.print(); obj2.print();
}
```

x = 25 , x = 25

- x is a non-static data member
- x cannot be shared between obj1 & obj2
- Non-static data members do not need separate definitions - instantiated with the object
- Non-static data members are initialized during object construction

Programming in Modern C++

static Data Member

```
#include<iostream>
using namespace std;
class MyClass { static int x; // Declare static
public:
    void get() { x = 15; }
    void print() { x = x + 10;
        cout << "x =" << x << endl;
    }
};

int MyClass::x = 0; // Define static data member
int main() {
    MyClass obj1, obj2; // Have same x
    obj1.get(); obj2.get();
    obj1.print(); obj2.print();
}
```

x = 25 , x = 35

- x is static data member
- x is shared by all MyClass objects including obj1 & obj2
- static data members must be defined in the global scope
- static data members are initialized during program start-up

Partha Pratim Das

M16.7



Program 16.02: static Data Member: Print Task (**Unsafe**)

Module M16

Partha Pratim
Das

Weekly Recap

Objectives &
Outline

static Data
Member

Example

Print Task

Order of Initialization

static Member
function

Print Task

Count Objects

Comparison

Singleton Class

Module Summary

```
#include <iostream>
using namespace std;
class PrintJobs { int nPages_; /* # of pages in current job */ public:
    static int nTrayPages_; /* # of pages in the tray */ static int nJobs_; // # of print jobs executing
    PrintJobs(int nP): nPages_(nP) { ++nJobs_; cout << "Printing " << nP << " pages" << endl;
        nTrayPages_ = nTrayPages_ - nP;
    } // Job started
    ~PrintJobs() { --nJobs_; } // Job done
};
int PrintJobs::nTrayPages_ = 500; // Definition and initialization -- load paper
int PrintJobs::nJobs_ = 0; // Definition and initialization -- no job to start with
int main() {
    cout << "Jobs = " << PrintJobs::nJobs_ << endl;
    cout << "Pages= " << PrintJobs::nTrayPages_ << endl;
    PrintJobs job1(10);
    cout << "Jobs = " << PrintJobs::nJobs_ << endl;
    cout << "Pages= " << PrintJobs::nTrayPages_ << endl;
    {
        PrintJobs job1(30), job2(20); // Different job1 in block scope
        cout << "Jobs = " << PrintJobs::nJobs_ << endl;
        cout << "Pages= " << PrintJobs::nTrayPages_ << endl;
        PrintJobs::nTrayPages_ += 100; // Load 100 more pages
    }
    cout << "Jobs = " << PrintJobs::nJobs_ << endl;
    cout << "Pages= " << PrintJobs::nTrayPages_ << endl;
}
```

Output:

```
Jobs = 0
Pages= 500
Printing 10 pages
Jobs = 1 // same nJobs_, nTrayPages_
Pages= 490
Printing 30 pages
Printing 20 pages
Jobs = 3 // same nJobs_, nTrayPages_
Pages= 440
Jobs = 1 // same nJobs_, nTrayPages_
Pages= 540
```




Program 16.03/04: Order of Initialization: Order of Definitions

Module M16

Partha Pratim Das

Weekly Recap

Objectives & Outline

static Data Member

Example

Print Task

Order of Initialization

static Member function

Print Task

Count Objects

Comparison

Singleton Class

Module Summary

```
#include <iostream>
#include <string>
using namespace std;
class Data { string id_; public:
    Data(const string& id) : id_(id)
    { cout << "Construct: " << id_ << endl; }
    ~Data()
    { cout << "Destruct: " << id_ << endl; }
};
class MyClass {
    static Data d1_; // Listed 1st
    static Data d2_; // Listed 2nd
};
Data MyClass::d1_("obj_1"); // Constructed 1st
Data MyClass::d2_("obj_2"); // Constructed 2nd

int main() { }
-----
Construct: obj_1
Construct: obj_2
Destruct: obj_2
Destruct: obj_1
```

```
#include <iostream>
#include <string>
using namespace std;
class Data { string id_; public:
    Data(const string& id) : id_(id)
    { cout << "Construct: " << id_ << endl; }
    ~Data()
    { cout << "Destruct: " << id_ << endl; }
};
class MyClass {
    static Data d2_; // Order of static members swapped
    static Data d1_;
};
Data MyClass::d1_("obj_1"); // Constructed 1st
Data MyClass::d2_("obj_2"); // Constructed 2nd

int main() { }
-----
Construct: obj_1
Construct: obj_2
Destruct: obj_2
Destruct: obj_1
```

- Order of initialization of **static** data members does not depend on their order in the definition of the class. It depends on the order their definition and initialization in the source



static Member function

Module M16

Partha Pratim
Das

Weekly Recap

Objectives &
Outline

static Data
Member

Example

Print Task

Order of Initialization

**static Member
function**

Print Task

Count Objects

Comparison

Singleton Class

Module Summary

static Member function



static Member Function

Module M16

Partha Pratim Das

Weekly Recap

Objectives & Outline

static Data Member

Example

Print Task

Order of Initialization

static Member function

Print Task

Count Objects

Comparison

Singleton Class

Module Summary

- A **static** member function
 - does not have **this** pointer – not associated with any object
 - *cannot access* non-**static** data members
 - *cannot invoke* non-**static** member functions
 - can be accessed
 - ▷ with the class-name followed by the scope resolution operator (**::**)
 - ▷ as a member of any object of the class
 - is needed to read / write **static** data members
 - ▷ Again, for encapsulation **static** data members should be **private**
 - ▷ **get()-set()** idiom is built for access (**static** member functions in **public**)
 - *may initialize static* data members *even before any object creation*
 - *cannot co-exist with a non-static version* of the same function
 - *cannot be declared* as **const**
- We repeat the Print Task with better (safer) modeling and coding



Program 16.05: static Data & Member Function: Print Task (Safe)

Module M16

Partha Pratim
Das

Weekly Recap

Objectives &
Outline

static Data
Member

Example

Print Task
Order of Initialization

static Member
function

Print Task
Count Objects

Comparison

Singleton Class

Module Summary

```
// #include <iostream> using namespace std;
class PrintJobs { int nPages_; // # of pages in current job
    static int nTrayPages_; /* # of pages in the tray */ static int nJobs_; // # of print jobs executing
public: PrintJobs(int nP) : nPages_(nP) { ++nJobs_; cout << "Printing " << nP << " pages" << endl;
    nTrayPages_ = nTrayPages_ - nP; } // Job started
    ~PrintJobs() { --nJobs_; } // Job done
    static int getJobs() { return nJobs_; } // get on nJobs_. Readonly. No set provided
    static int checkPages() { return nTrayPages_; } // get on nTrayPages_
    static void loadPages(int nP) { nTrayPages_ += nP; } // set on nTrayPages_
};
int PrintJobs::nTrayPages_ = 500; // Definition and initialization -- load paper
int PrintJobs::nJobs_ = 0; // Definition and initialization -- no job to start with
int main() { cout << "Jobs = " << PrintJobs::getJobs() << endl;
    cout << "Pages= " << PrintJobs::checkPages() << endl;
    PrintJobs job1(10);
    cout << "Jobs = " << PrintJobs::getJobs() << endl;
    cout << "Pages= " << PrintJobs::checkPages() << endl;
    {
        PrintJobs job1(30), job2(20); // Different job1 in block scope
        cout << "Jobs = " << PrintJobs::getJobs() << endl;
        cout << "Pages= " << PrintJobs::checkPages() << endl;
        PrintJobs::loadPages(100); // Load 100 more pages
    }
    cout << "Jobs = " << PrintJobs::getJobs() << endl;
    cout << "Pages= " << PrintJobs::checkPages() << endl;
}
```

Programming in Modern C++

Partha Pratim Das

Output:

```
Jobs = 0
Pages= 500
Printing 10 pages
Jobs = 1 // same nJobs_, nTrayPages_
Pages= 490
Printing 30 pages
Printing 20 pages
Jobs = 3 // same nJobs_, nTrayPages_
Pages= 440
Jobs = 1 // same nJobs_, nTrayPages_
Pages= 540
```

M16.12



Counting Objects

Module M16

Partha Pratim Das

Weekly Recap

Objectives & Outline

static Data Member

Example

Print Task

Order of Initialization

static Member function

Print Task

Count Objects

Comparison

Singleton Class

Module Summary

- We illustrate another example and use for `static` data member and member function
 - Here we want to track the number of objects created and destroyed for a class at any point in the program
 - Naturally no object can keep this information. So we hold two `static` data members
 - ▷ `nObjCons_`: Number of objects created since beginning. It is *read-only and incremented in every constructor*
 - ▷ `nObjDes_`: Number of objects destroyed since beginning. It is *read-only and incremented in the destructor*
 - At any point (`nObjCons_ - nObjDes_`) gives the number of *Live* objects
 - In an alternate (less informative model) we may just maintain `static` data member `nLive_` which is *incremented in every constructor and decremented in the destructor*



Program 16.06: Count Objects

Module M16

Partha Pratim Das

Weekly Recap

Objectives & Outline

static Data Member

Example

Print Task

Order of Initialization

static Member function

Print Task

Count Objects

Comparison

Singleton Class

Module Summary

```
#include <iostream>
#include <string>
using namespace std;
class MyClass { string id_; // Object ID
    static int nObjCons_, nObjDes_; // Object history
public:
    MyClass(const string& id) : id_(id)
    { ++nObjCons_;
      cout << "ctor: " << id_ << " "; getObjLive(); }
    ~MyClass() { ++nObjDes_;
      cout << "dtor: " << id_ << " "; getObjLive(); }
    static int getObjConstructed()
    { return nObjCons_; }
    static int getObjDestructed()
    { return nObjDes_; }
    // Get number of live objects
    static int getObjLive() {
        int nLive = nObjCons_ - nObjDes_;
        cout << "Live Objects = " << nLive << endl;
        return nLive;
    }
};
int MyClass::nObjCons_ = 0;
int MyClass::nObjDes_ = 0;
```

```
int dummy1(MyClass::getObjLive()); // Before (main())
MyClass sObj("sObj");
int dummy2(MyClass::getObjLive()); // Before (main())
int main() { MyClass::getObjLive();
    MyClass aObj("aObj");
    MyClass *dObj = new MyClass("dObj");
    {
        MyClass bObj("bObj");
        delete dObj;
    }
    MyClass::getObjLive();
}
```

```
Live Objects = 0 // Before any object (dummy1)
ctor: sObj Live Objects = 1
Live Objects = 1 // Before main() (dummy2)
Live Objects = 1 // Enter main()
ctor: aObj Live Objects = 2
ctor: dObj Live Objects = 3
ctor: bObj Live Objects = 4
dtor: dObj Live Objects = 3
dtor: bObj Live Objects = 2
Live Objects = 2 // Exit main()
: aObj Live Objects = 1
dtor: sObj Live Objects = 0 // After all objectst
```



static vis-a-vis non-static

Module M16

Partha Pratim
Das

Weekly Recap

Objectives &
Outline

static Data
Member

Example

Print Task

Order of Initialization

static Member
function

Print Task

Count Objects

Comparison

Singleton Class

Module Summary

NPTEL

static vis-a-vis non-static



Comparison of static vis-a-vis non-static

Module M16

Partha Pratim Das

Weekly Recap

Objectives & Outline

static Data Member

Example

Print Task

Order of Initialization

static Member function

Print Task

Count Objects

Comparison

Singleton Class

Module Summary

static Data Members

- Declared using keyword **static**
- All objects of a class share the *same copy / instance*
- *Accessed* using the *class name or object*
- May be **public** or **private**
- Belongs to the **namespace** of the class
- May be **const**
- Are *constructed before* **main()** is invoked
- Are *destroyed after* (in reverse order) **main()** returns
- Are *constructed* in the order of definitions in source
- Has a *lifetime* encompassing **main()**
- *Allocated* in **static** memory

static Member Functions

- Declared using keyword **static**
- *Has no* **this** pointer parameter
- *Invoked* using the *class name or object*
- May be **public** or **private**
- Belongs to the **namespace** of the class
- *Can access* **static** data members and methods
- *Cannot access* non-**static** data members or methods
- Can be invoked anytime during program execution
- Cannot be **virtual** or **const**
- *Constructor* is **static** though not declared **static**

Non-static Data Members

- Declared *without* using keyword **static**
- Each object of the class gets its *own copy / instance*
- *Accessed* only through an *object* of the class
- May be **public** or **private**
- Belongs to the **namespace** of the class
- May be **const**
- Are *constructed during* object construction
- Are *destroyed during* object destruction
- Are *constructed* in the order of listing in the class
- Has a *lifetime* as of the lifetime of the object
- *Allocated* in **static**, **stack**, or **heap** memory as of the object

Non-static Member Functions

- Declared *without* using keyword **static**
- *Has an implicit* **this** pointer parameter
- *Invoked* only through an *object* of the class
- May be **public** or **private**
- Belongs to the **namespace** of the class
- *Can access* **static** data members and methods
- *Can access* non-**static** data members and methods
- Can be invoked only during *lifetime* of the object
- May be **virtual** and / or **const**
- *There cannot be* a non-**static** Constructor



Singleton Class

Module M16

Partha Pratim
Das

Weekly Recap

Objectives &
Outline

static Data
Member

Example

Print Task

Order of Initialization

static Member
function

Print Task

Count Objects

Comparison

Singleton Class

Module Summary

Singleton Class



Singleton Class

Module M16

Partha Pratim Das

Weekly Recap

Objectives & Outline

static Data Member

Example

Print Task

Order of Initialization

static Member function

Print Task

Count Objects

Comparison

Singleton Class

Module Summary

- **Singleton** is a *creational design pattern*
 - ensures that *only one object* of its kind exists and
 - provides a *single point of access* to it for any other code
- A class is called a Singleton if it satisfies the above conditions
- Many classes are singleton:
 - President of India
 - Prime Minister of India
 - Director of IIT Kharagpur
 - CEO of a Company
 - ...
- How to implement a Singleton Class?
- How to restrict that user can created *only one* instance?



Program 16.07: static Data & Member Function Singleton Printer

Module M16

Partha Pratim
Das

Weekly Recap

Objectives &
Outline

static Data
Member

Example

Print Task

Order of Initialization

static Member
function

Print Task

Count Objects

Comparison

Singleton Class

Module Summary

```
#include <iostream>
using namespace std;

class Printer { /* THIS IS A SINGLETON PRINTER -- ONLY ONE INSTANCE */
private: bool blackAndWhite_, bothSided_;
    Printer(bool bw = false, bool bs = false) : blackAndWhite_(bw), bothSided_(bs)
    { cout << "Printer constructed" << endl; } // Private -- Printer cannot be constructed!
    static Printer *myPrinter_; // Pointer to the Instance of the Singleton Printer
public: ~Printer() { cout << "Printer destructed" << endl; }
    static const Printer& printer(bool bw = false, bool bs = false) { // Access the Printer
        if (!myPrinter_) myPrinter_ = new Printer(bw, bs); // Constructed for first call
        return *myPrinter_; // Reused from next time
    }
    void print(int nP) const { cout << "Printing " << nP << " pages" << endl; }
};

Printer *Printer::myPrinter_ = 0;

int main() {
    Printer::printer().print(10);
    Printer::printer().print(20);

    delete &Printer::printer();
}
```

Output:

```
Printer constructed
Printing 10 pages
Printing 20 pages
Printer destructed
```



Program 16.08: Using function-local static Data Singleton Printer

Module M16

Partha Pratim Das

Weekly Recap

Objectives & Outline

static Data Member

Example

Print Task

Order of Initialization

static Member function

Print Task

Count Objects

Comparison

Singleton Class

Module Summary

```
#include <iostream>
using namespace std;

class Printer { /* THIS IS A SINGLETON PRINTER -- ONLY ONE INSTANCE */
    bool blackAndWhite_, bothSided_;
    Printer(bool bw = false, bool bs = false) : blackAndWhite_(bw), bothSided_(bs)
    { cout << "Printer constructed" << endl; }
    ~Printer() { cout << "Printer destructed" << endl; }
public:
    static const Printer& printer(bool bw = false, bool bs = false) {
        static Printer myPrinter(bw, bs); // The Singleton -- constructed the first time

        return myPrinter;
    }
    void print(int nP) const { cout << "Printing " << nP << " pages" << endl; }
};

int main() {
    Printer::printer().print(10);
    Printer::printer().print(20);
}
```

- Function local **static** object is used
- No memory management overhead – so destructor too get **private**
- This is called **Meyer's Singleton**

Output:

```
Printer constructed
Printing 10 pages
Printing 20 pages
Printer destructed
```



Module Summary

Module M16

Partha Pratim Das

Weekly Recap

Objectives & Outline

static Data Member

Example

Print Task

Order of Initialization

static Member function

Print Task

Count Objects

Comparison

Singleton Class

Module Summary

- Introduced `static` data member
- Introduced `static` member function
- Exposed to use of static members
- Singleton Class discussed



Module M17

Partha Pratim
Das

Module Recap

Objectives &
Outlines

friend Function

Matrix-Vector
Multiplication

Linked List

friend Class

Linked List
Iterator

Properties

Comparison

Module Summary

Programming in Modern C++

Module M17: friend Function and friend Class

Partha Pratim Das

Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur

ppd@cse.iitkgp.ac.in

All url's in this module have been accessed in September, 2021 and found to be functional



Module Recap

Module M17

Partha Pratim
Das

Module Recap

Objectives &
Outlines

friend Function

Matrix-Vector
Multiplication

Linked List

friend Class

Linked List

Iterator

Properties

Comparison

Module Summary

- Introduced `static` data member
- Introduced `static` member function
- Exposed to use of static members
- Singleton Class discussed



Module Recap

Module M17

Partha Pratim
Das

Module Recap

Objectives &
Outlines

friend Function

Matrix-Vector
Multiplication

Linked List

friend Class

Linked List

Iterator

Properties

Comparison

Module Summary

- Introduced `static` data member
- Introduced `static` member function
- Exposed to use of static members
- Singleton Class discussed



Module Objectives

Module M17

Partha Pratim
Das

Module Recap

Objectives &
Outlines

friend Function

Matrix-Vector
Multiplication

Linked List

friend Class

Linked List

Iterator

Properties

Comparison

Module Summary

- Understand `friend` function and class

NPTEL



Module Outline

Module M17

Partha Pratim Das

Module Recap

Objectives & Outlines

friend Function

Matrix-Vector Multiplication

Linked List

friend Class

Linked List

Iterator

Properties

Comparison

Module Summary

1 Module Recap

2 friend Function

- Matrix-Vector Multiplication
- Linked List

3 friend Class

- Linked List
- Iterator

4 Properties of friend

5 Comparison

6 Module Summary



friend Function

Module M17

Partha Pratim
Das

Module Recap

Objectives &
Outlines

friend Function

Matrix-Vector
Multiplication

Linked List

friend Class

Linked List

Iterator

Properties

Comparison

Module Summary

friend Function



Program 17.01: friend function: Basic Notion

Module M17

Partha Pratim Das

Module Recap

Objectives & Outlines

friend Function

Matrix-Vector Multiplication

Linked List

friend Class

Linked List

Iterator

Properties

Comparison

Module Summary

Ordinary function

```
#include<iostream>
using namespace std;
class MyClass { int data_;
public:
    MyClass(int i) : data_(i) { }
};
void display(const MyClass& a) { // gbl. func.
    cout << "data = " << a.data_; // Error 1
}
int main() {
    MyClass obj(10);

    display(obj);
}
```

- `display()` is a non-member function
- **Error 1:** '`MyClass::data_`' : cannot access private member declared in class '`MyClass`'

friend function

```
#include<iostream>
using namespace std;
class MyClass { int data_;
public:
    MyClass(int i) : data_(i) { }
    friend void display(const MyClass& a);
};
void display(const MyClass& a) { // global function
    cout << "data = " << a.data_; // Okay
}
int main() {
    MyClass obj(10);

    display(obj);
}
```

- `display()` is a non-member function; but friend to class `MyClass`
- Able to access `data_` even though it is private in class `MyClass`
- **Output:** `data = 10`



friend function

Module M17

Partha Pratim Das

Module Recap

Objectives & Outlines

friend Function

Matrix-Vector Multiplication

Linked List

friend Class

Linked List

Iterator

Properties

Comparison

Module Summary

- A **friend** function of a class
 - has access to the **private** and **protected** members of the class (*breaks the encapsulation*) in addition to **public** members
 - must have its *prototype included within the scope of the class* prefixed with the keyword **friend**
 - *does not have its name qualified* with the class scope
 - *is not called with an invoking object* of the class
 - *can be declared friend* in *more than one classes*
- A **friend** function can be a
 - *global function*
 - a *member function* of a class
 - a *function template*



Program 17.02: Multiply a Matrix with a Vector

Module M17

Partha Pratim Das

Module Recap

Objectives & Outlines

friend Function

Matrix-Vector Multiplication

Linked List

friend Class

Linked List

Iterator

Properties

Comparison

Module Summary

```
#include <iostream>
using namespace std;

class Matrix; // Forward declaration

class Vector { int e_[3]; int n_; public:
    Vector(int n) : n_(n) {
        for (int i = 0; i < n_; ++i) // Arbitrary
            e_[i] = i + 1;           // init.
    }
    void Clear() { // Set a zero vector
        for(int i = 0; i < n_; ++i)
            e_[i] = 0;
    }
    void Show() { // Show the vector
        for(int i = 0; i < n_; ++i)
            cout << e_[i] << " ";
        cout << endl << endl;
    }
    friend Vector Prod(Matrix *pM, Vector *pV);
};
```

```
class Matrix { int e_[3][3]; int m_, n_; public:
    Matrix(int m, int n) : m_(m), n_(n) { // Arbitrary
        for(int i = 0; i < m_; ++i)       // init.
            for(int j = 0; j < n_; ++j) e_[i][j] = i + j;
    }
    void Show() { // Show the matrix
        for (int i = 0; i < m_; ++i) {
            for (int j = 0; j < n_; ++j)
                cout << e_[i][j] << " ";
            cout << endl;
        } cout << endl;
    }
    friend Vector Prod(Matrix *pM, Vector *pV);
};

Vector Prod(Matrix *pM, Vector *pV) {
    Vector v(pM->m_); v.Clear();
    for(int i = 0; i < pM->m_; i++)
        for(int j = 0; j < pM->n_; j++)
            v.e_[i] += pM->e_[i][j] * pV->e_[j];
    return v;
}
```

- `Vector Prod(Matrix*, Vector*);` is a **global function**
- `Vector Prod(Matrix*, Vector*);` is **friend** of class **Vector** as well as class **Matrix**
- This function accesses the **private** data members of both these classes



Program 17.02: Multiply a Matrix with a Vector

Module M17

Partha Pratim Das

Module Recap

Objectives & Outlines

friend Function

Matrix-Vector Multiplication

Linked List

friend Class

Linked List

Iterator

Properties

Comparison

Module Summary

```
int main() {  
    Matrix M(2, 3);  
    Vector V(3);  
  
    Vector PV = Prod(&M, &V);  
  
    M.Show();  
    V.Show();  
    PV.Show();  
  
    return 0;  
}
```

Output:

```
0 1 2    // Matrix M  
1 2 3  
  
1 2 3    // Vector V  
  
8 14     // Product Vector PV
```

- `Vector Prod(Matrix*, Vector*);` is a **global function**
- `Vector Prod(Matrix*, Vector*);` is **friend** of class `Vector` as well as class `Matrix`
- This function accesses the **private** data members of both these classes



Program 17.03: Linked List

Module M17

Partha Pratim Das

Module Recap

Objectives & Outlines

friend Function

Matrix-Vector Multiplication

Linked List

friend Class

Linked List

Iterator

Properties

Comparison

Module Summary

```
#include <iostream>
using namespace std;

class Node;    // Forward declaration
class List {
    Node *head; // Head of the list
    Node *tail; // Tail of the list
public:
    List(Node *h = 0): head(h), tail(h) { }
    void display();
    void append(Node *p);
};

class Node {
    int info; // Data of the node
    Node *next; // Ptr. to next node
public:
    Node(int i): info(i), next(0) { }
    friend void List::display();
    friend void List::append(Node *);
};
```

```
void List::display() { // friend of Node
    Node *ptr = head;
    while (ptr) { cout << ptr->info << " ";
        ptr = ptr->next;
    }
}

void List::append(Node *p) { // friend of Node
    if (!head) head = tail = p;
    else {
        tail->next = p;
        tail = tail->next;
    }
}

int main() { List l; // Init. null list
    Node n1(1), n2(2), n3(3); // Few nodes
    l.append(&n1); // Add nodes to list
    l.append(&n2);
    l.append(&n3);
    l.display(); // Show list
}
```

- List is built on Node. Hence List needs to know the internals of Node
- void List::append(Node *); needs the internals of Node – hence friend member function is used
- void List::display(); needs the internals of Node – hence friend member function is used
- We can do better with friend classes



friend Class

Module M17

Partha Pratim
Das

Module Recap

Objectives &
Outlines

friend Function

Matrix-Vector
Multiplication

Linked List

friend Class

Linked List

Iterator

Properties

Comparison

Module Summary

NPTEL

friend Class



friend class

Module M17

Partha Pratim Das

Module Recap

Objectives &
Outlines

friend Function

Matrix-Vector
Multiplication

Linked List

friend Class

Linked List

Iterator

Properties

Comparison

Module Summary

- A **friend** class of a class
 - has access to the **private** and **protected** members of the class (*breaks the encapsulation*) in addition to **public** members
 - *does not have its name qualified* with the class scope (*not a nested class*)
 - *can be declared friend* in *more than one classes*
- A **friend** class can be a
 - **class**
 - **class template**



Program 17.04: Linked List

Module M17

Partha Pratim Das

Module Recap

Objectives & Outlines

friend Function

Matrix-Vector Multiplication

Linked List

friend Class

Linked List

Iterator

Properties

Comparison

Module Summary

```
#include <iostream>
using namespace std;

class Node; // Forward declaration
class List {
    Node *head; // Head of the list
    Node *tail; // Tail of the list
public:
    List(Node *h = 0): head(h), tail(h) { }
    void display();
    void append(Node *p);
};

class Node {
    int info; // Data of the node
    Node *next; // Ptr to next node
public:
    Node(int i): info(i), next(0) { }
    // friend void List::display();
    // friend void List::append(Node *);
    friend class List;
};
```

```
void List::display() {
    Node *ptr = head;
    while (ptr) { cout << ptr->info << " ";
        ptr = ptr->next;
    }
}

void List::append(Node *p) {
    if (!head) head = tail = p;
    else {
        tail->next = p;
        tail = tail->next;
    }
}

int main() { List l; // Init null list
    Node n1(1), n2(2), n3(3); // Few nodes
    l.append(&n1); // Add nodes to list
    l.append(&n2);
    l.append(&n3);

    l.display(); // Show list
}
```

- List class is now a friend of Node class. Hence it has full visibility into the internals of Node
- When multiple member functions need to be friends, it is better to use friend class



Program 17.05: Linked List with Iterator

Module M17

Partha Pratim Das

Module Recap

Objectives & Outlines

friend Function

Matrix-Vector Multiplication

Linked List

friend Class

Linked List

Iterator

Properties

Comparison

Module Summary

```
#include <iostream>
using namespace std;
class Node; class List; // Forward declarations
class Iterator { Node *node; // Current Node
    List *list; // Current List
public: Iterator() : node(0), list(0) { }
    void begin(List *); // Init
    bool end(); // Check end
    void next(); // Go to next
    int data(); // Get node data
};
class List { Node *head, *tail; public:
    List(Node *h=0): head(h), tail(h) { }
    void append(Node *p);
    friend class Iterator;
};
class Node { int info; Node *next; public:
    Node(int i) : info(i), next(0) { }
    friend class List;
    friend class Iterator;
};
```

- An **Iterator** now traverses over the elements of the **List**
- **void List::display()** is dropped from **List** and can be written in **main()**
- **List** class is a **friend** of **Node** class
- **Iterator** class is a **friend** of **List** and **Node** classes

Programming in Modern C++

```
// Iterator methods
void Iterator::begin(List *l) {
    list = l; node = l->head; // Set list & Init
}
bool Iterator::end() { return node == 0; }
void Iterator::next() { node = node->next; }
int Iterator::data() { return node->info; }

void List::append(Node *p) {
    if (!head) head = tail = p;
    else { tail->next = p; tail = tail->next; }
}

int main() { List l;
    Node n1(1), n2(2), n3(3);
    l.append(&n1); l.append(&n2); l.append(&n3);

    Iterator i;
    for(i.begin(&l); !i.end(); i.next()) {
        cout << i.data() << " "; // Iteration Loop
    }
}
```

Partha Pratim Das

M17.15



Properties of friend

Module M17

Partha Pratim
Das

Module Recap

Objectives &
Outlines

friend Function

Matrix-Vector
Multiplication

Linked List

friend Class

Linked List

Iterator

Properties

Comparison

Module Summary

Properties of friend



Properties of friend

Module M17

Partha Pratim Das

Module Recap

Objectives & Outlines

friend Function

Matrix-Vector Multiplication

Linked List

friend Class

Linked List

Iterator

Properties

Comparison

Module Summary

- **friend**ship is neither *commutative* nor *transitive*
 - A is a **friend** of B *does not imply* that B is a **friend** of A
 - A is a **friend** of B and B is a **friend** of C *does not imply* that A is a **friend** of C
- **Visibility and Encapsulation**
 - **public**: a declaration that is accessible to all
 - **protected**: a declaration that is accessible only to the class itself and its subclasses
 - **private**: a declaration that is accessible only to the class itself
 - **friend**: a declaration that is accessible only to **friend**'s of a class. **friends** tend to *break data hiding* and **must be used judiciously**. Like:
 - ▷ A function needs to access the internals of two (or more) independent classes (Matrix-Vector Multiplication)
 - ▷ A class is built on top of another (List-Node Access, List Iterator)
 - ▷ Certain situations of operator overloading (like streaming operators)



Comparison of friend vis-a-vis Member Functions

Module M17

Partha Pratim
Das

Module Recap

Objectives &
Outlines

friend Function

Matrix-Vector
Multiplication

Linked List

friend Class

Linked List

Iterator

Properties

Comparison

Module Summary

NPTEL

Comparison of friend vis-a-vis Member Functions



Comparison of friend vis-a-vis Member Functions

Module M17

Partha Pratim Das

Module Recap

Objectives & Outlines

friend Function

Matrix-Vector Multiplication

Linked List

friend Class

Linked List

Iterator

Properties

Comparison

Module Summary

friend Functions

- Declared using the keyword **friend**
- Declared in *one or more classes*
- *Not a part* of the class, *not defined* in the **namespace** of the classes
- Has *access to all* **private**, **public**, and **protected** members of classes
- May be *global* or *member function* of some other class
- Called with an *object* (non-**static** member), an *object* / a *class* (**static** member), or as a *global* function
- Does not have **this** pointer (of the class it accesses). Needs the *pointer to the object*
- Used in *collaborative multi-class design*
- Breaks *encapsulation*
- Binary operation usually takes two explicit parameters
- Unary operator takes at least one explicit parameter

static & Non-static Member Functions

- Declared in **private**, **public**, or **protected** specifier
- Declared *only in scope of a particular class*
- *Part* of the class definition, *defined* in the **namespace** of the class
- Has *access to all* **private**, **public**, and **protected** members of its class, if non-**static**
- Has *access to only* **private**, **public**, and **protected static** members of its class, if **static**
- *Member function* of the class
- Called with an *object* (non-**static** member) or an *object* / a *class* (**static** member) of the defining class
- Has **this** pointer of the defining class, if a Non-**static** and no **this** pointer if **static**
- Used for *modularity* and *encapsulation*
- Ensures *encapsulation*
- Binary operations usually take only one explicit parameter
- Unary operator does not take any explicit parameter



Module Summary

Module M17

Partha Pratim Das

Module Recap

Objectives & Outlines

friend Function

Matrix-Vector Multiplication

Linked List

friend Class

Linked List

Iterator

Properties

Comparison

Module Summary

- Introduced the notion of `friend` function
- Introduced the notion of `friend` class
- Studied the use of `friend` function and `friend` class with examples
- `friend` introduces visibility hole by breaking encapsulation – should be used with care



Module M18

Partha Pratim
Das

Objectives &
Outlines

Operator
Function

Non-Member

Member

Rules

Global Function

`public` data
members

`private` data
members

Member Function

`operator+`

`operator=`

Unary Operators

Module Summary

Programming in Modern C++

Module M18: Overloading Operator for User-Defined Types: Part 1

Partha Pratim Das

Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur

ppd@cse.iitkgp.ac.in

All url's in this module have been accessed in September, 2021 and found to be functional



Module Recap

Module M18

Partha Pratim
Das

Objectives & Outlines

Operator Function

Non-Member

Member

Rules

Global Function

public data
members

private data
members

Member Function

operator+

operator=

Unary Operators

Module Summary

- Introduced the notion of `friend` function
- Introduced the notion of `friend` class
- Studied the use of `friend` function and `friend` class with examples
- `friend` introduces visibility hole by breaking encapsulation – should be used with care



Module Objectives

Module M18

Partha Pratim Das

Objectives & Outlines

Operator Function

Non-Member

Member

Rules

Global Function

public data members

private data members

Member Function

operator+

operator=

Unary Operators

Module Summary

- Understand how to overload operators for a user-defined type (class)
- Understand the aspects of overloading by global function and member function



Module Outline

Module M18

Partha Pratim Das

Objectives & Outlines

Operator Function

Non-Member

Member

Rules

Global Function

public data members

private data members

Member Function

operator+

operator=

Unary Operators

Module Summary

- 1 Operator Function
 - Non-Member Function
 - Member Function
 - Operator Overloading Rules
- 2 Using Global Function
 - public data members
 - private data members
- 3 Using Member Function
 - operator+
 - operator=
 - Unary Operators
- 4 Module Summary



Operator Function

Module M18

Partha Pratim
Das

Objectives &
Outlines

**Operator
Function**

Non-Member

Member

Rules

Global Function

`public` data
members

`private` data
members

Member Function

`operator+`

`operator=`

Unary Operators

Module Summary

Operator Function



How can operator functions help?

Module M18

Partha Pratim Das

Objectives & Outlines

Operator Function

Non-Member

Member

Rules

Global Function

public data members

private data members

Member Function

operator+

operator=

Unary Operators

Module Summary

- We have seen how **overloading operator+** a C-string wrapped in struct allows us a compact notation for concatenation of two strings (**Module 09**)
- We have seen how **overloading operator=** can define the deep / shallow copy for a UDT and / or help with user-defined copy semantics (**Module 14**)
- In general, operator overloading helps us to **build complete algebra** for UDT's much in the same line as is available for built-in types:
 - **Complex type**: Add (+), Subtract (-), Multiply (*), Divide (/), Conjugate (!), Compare (==, !=, ...), etc.
 - **Fraction type**: Add (+), Subtract (-), Multiply (*), Divide (/), Normalize (unary *), Compare (==, !=, ...), etc.
 - **Matrix type**: Add (+), Subtract (-), Multiply (*), Divide (/), Invert (!), Compare (==), etc.
 - **Set type**: Union (+), Difference (-), Intersection (*), Subset (< <=), Superset (> >=), Compare (==, !=), etc.
 - **Direct IO**: read (<<) and write (>>) for all types
- Advanced examples include:
 - **Smart Pointers**: De-reference (unary *), Indirection (->), Copy (=), Compare (==, !=), etc.
 - **Function Objects or Functors**: Invocation (())



Operator Functions in C++: RECAP (Module 9)

Module M18

Partha Pratim Das

Objectives & Outlines

Operator Function

Non-Member

Member

Rules

Global Function

public data members

private data members

Member Function

operator+

operator=

Unary Operators

Module Summary

- Introduces a new keyword: `operator`
- Every operator is associated with an operator function that defines its behavior

Operator Expression	Operator Function
<code>a + b</code>	<code>operator+(a, b)</code>
<code>a = b</code>	<code>operator=(a, b)</code>
<code>c = a + b</code>	<code>operator=(c, operator+(a, b))</code>

- Operator functions are implicit for predefined operators of built-in types and cannot be redefined
- An operator function may have a signature as:

```
MyType a, b; // An enum or struct
```

```
// Operator function
```

```
MyType operator+(const MyType&, const MyType&);
```

```
a + b // Calls operator+(a, b)
```

- C++ allows users to define an operator function and overload it



Non-Member Operator Function

Module M18

Partha Pratim
Das

Objectives &
Outlines

Operator
Function

Non-Member

Member
Rules

Global Function

public data
members

private data
members

Member Function
operator+

operator=

Unary Operators

Module Summary

- A non-member operator function may be a

- Global Function
- **friend** Function

- **Binary Operator:**

```
MyType a, b; // An enum, struct or class
MyType operator+(const MyType&, const MyType&); // Global
friend MyType operator+(const MyType&, const MyType&); // Friend
```

- **Unary Operator:**

```
MyType operator++(const MyType&); // Global
friend MyType operator++(const MyType&); // Friend
```

- **Note:** The parameters may not be constant and may be passed by value. The return may also be by reference and may be constant

- **Examples:**

Operator Expression	Operator Function
a + b	operator+(a, b)
a = b	operator=(a, b)
++a	operator++(a)
a++	operator++(a, int) Special Case
c = a + b	operator=(c, operator+(a, b))



Member Operator Function

Module M18

Partha Pratim Das

Objectives & Outlines

Operator Function

Non-Member

Member Rules

Global Function

public data members

private data members

Member Function operator+

operator=

Unary Operators

Module Summary

- **Binary Operator:**

```
MyType a, b; // MyType is a class
MyType operator+(const MyType&); // Operator function
```

- The left operand is the invoking object – right is taken as a parameter

- **Unary Operator:**

```
MyType operator-(); // Operator function for Unary minus
MyType operator++(); // For Pre-Incrementer
MyType operator++(int); // For post-Incrementer
```

- The only operand is the invoking object
- **Note:** The parameters may not be constant and may be passed by value. The return may also be by reference and may be constant
- **Examples:**

Operator Expression	Operator Function
a + b	a.operator+(b)
a = b	a.operator=(b)
++a	a.operator++()
a++	a.operator++(int) // Special Case
c = a + b	c.operator =(a.operator+(b))



Operator Overloading – Summary of Rules: RECAP (Module 9)

Module M18

Partha Pratim Das

Objectives & Outlines

Operator Function

Non-Member

Member

Rules

Global Function

public data members

private data members

Member Function

operator+

operator=

Unary Operators

Module Summary

- No new operator such as `**`, `<>`, or `&|` can be defined for overloading
- Intrinsic properties of the overloaded operator cannot be change
 - *Preserves arity*
 - *Preserves precedence*
 - *Preserves associativity*
- These operators *can be overloaded*:
`[] + - * / % ^ & | ~ ! = += -= *= /= %= ^= &= |=`
`<< >> >>= <<= == != < > <= >= && || ++ -- , ->* -> ( ) [ ]`
- The operators `::` (scope resolution), `.` (member access), `.*` (member access through pointer to member), `sizeof`, and `?:` (ternary conditional) *cannot be overloaded*
- The overloads of operators `&&`, `||`, and `,` (comma) *lose their special properties*: short-circuit evaluation and sequencing
- The overload of `operator->` must either return a raw pointer or return an object (by reference or by value), for which `operator->` is in turn overloaded
- For a member operator function, invoking object is passed implicitly as the left operand but the right operand is passed explicitly
- For a non-member operator function (Global/friend) operands are always passed explicitly



Using Global Function

Module M18

Partha Pratim
Das

Objectives &
Outlines

Operator
Function

Non-Member

Member

Rules

Global Function

public data
members

private data
members

Member Function

operator+

operator=

Unary Operators

Module Summary

Using Global Function



Program 18.01: Using Global Function: public Data members (Unsafe)

Module M18

Partha Pratim Das

Objectives & Outlines

Operator Function

Non-Member

Member

Rules

Global Function

public data members

private data members

Member Function
operator+
operator=

Unary Operators

Module Summary

Overloading + for complex addition

```
#include <iostream>
using namespace std;
struct complx { // public data member
    double re, im;
};
complx operator+ (complx &a, complx &b) {
    complx r;
    r.re = a.re + b.re;
    r.im = a.im + b.im;
    return r;
}
int main() { complx d1 , d2 , d;
    d1.re = 10.5; d1.im = 12.25;
    d2.re = 20.5; d2.im = 30.25;
    d = d1 + d2; // Overload operator +
    cout << "Real:" << d.re << ", ";
    cout << "Imag:" << d.im;
}
```

● **Output:** Real: 31, Imag: 42.5

● **operator+** is overloaded to perform addition of two complex numbers which are of **struct complx** type

Overloading + for string cat

```
#include <iostream>
#include <cstdlib>
#include <cstring>
using namespace std;
typedef struct _String { char *str; } String;
String operator+(const String& s1, const String& s2) {
    String s;
    s.str = (char *) malloc(strlen(s1.str) +
                             strlen(s2.str) + 1);
    strcpy(s.str, s1.str); strcat(s.str, s2.str);
    return s;
}
int main() { String fName, lName, name;
    fName.str = strdup("Partha ");
    lName.str = strdup("Das");
    name = fName + lName; // Overload operator +
    cout << "First Name: " << fName.str << endl;
    cout << "Last Name: " << lName.str << endl;
    cout << "Full Name: " << name.str << endl;
}
```

● **Output:** First Name: Partha, Last Name: Das, Full name: Partha Das

● **operator+** is overloaded to perform concat of first name and last to form full name. The data type is **String**



Program 18.02: Using Global Function: private Data members (Safe)

Module M18

Partha Pratim Das

Objectives & Outlines

Operator Function

Non-Member

Member

Rules

Global Function

public data members

private data members

Member Function

operator+

operator=

Unary Operators

Module Summary

```
#include <iostream>
using namespace std;
class Complex { // Private data members
    double re, im;
public:
    Complex(double a=0.0, double b=0.0):
        re(a), im(b) { } ~Complex() { }
    void display();
    double real() { return re; }
    double img() { return im; }
    double set_real(double r) { re = r; }
    double set_img(double i) { im = i; }
};
void Complex::display() {
    cout << re << " +j " << im << endl;
}
```

```
Complex operator+(Complex &t1, Complex &t2) {
    Complex sum;
    sum.set_real(t1.real() + t2.real());
    sum.set_img(t1.img() + t2.img());
    return sum;
}
int main() {
    Complex c1(4.5, 25.25), c2(8.3, 10.25), c3;
    cout << "1st complex No: "; c1.display();
    cout << "2nd complex No: "; c2.display();
    c3 = c1 + c2; // Overload operator +
    cout << "Sum = "; c3.display();
}
```

• Output:

1st complex No: 4.5 +j 25.25

2nd complex No: 8.3 +j 10.25

Sum = 12.8 +j 35.5

- Accessing private data members inside operator functions is clumsy
- Critical data members need to be exposed (get/set) violating encapsulation
- **Solution:** Member operator function or friend operator function



Using Member Function

Module M18

Partha Pratim
Das

Objectives &
Outlines

Operator
Function

Non-Member

Member

Rules

Global Function

public data
members

private data
members

Member Function

operator+

operator=

Unary Operators

Module Summary

Using Member Function



Program 18.03: Using Member Function

Module M18

Partha Pratim Das

Objectives & Outlines

Operator Function

Non-Member

Member

Rules

Global Function

public data members

private data members

Member Function

operator+

operator=

Unary Operators

Module Summary

```
#include <iostream>
using namespace std;
class Complex { // Private data members
    double re, im;
public:
    Complex(double a=0.0, double b=0.0):
        re(a), im(b) { } ~Complex() { }
    void display();
    Complex operator+(const Complex &c) {
        Complex r;
        r.re = re + c.re;
        r.im = im + c.im;
        return r;
    }
};
```

```
void Complex::display() {
    cout << re;
    cout << " +j " << im << endl;
}
int main() {
    Complex c1(4.5, 25.25), c2(8.3, 10.25), c3;
    cout << "1st complex No:";
    c1.display();
    cout << "2nd complex No:";
    c2.display();
    c3 = c1 + c2; // Overloaded operator +
    cout << "Sum = ";
    c3.display();
    return 0;
}
```

• Output:

1st complex No: 4.5 +j 25.25

2nd complex No: 8.3 +j 10.25

Sum = 12.8 +j 35.5

- Performing `c1 + c2` is equivalent to `c1.operator+(c2)`
- `c1` invokes the `operator+` function and `c2` is passed as an argument
- Similarly we can implement all binary operators (`%`, `-`, `*`, etc..)
- **Note:** No need of two arguments in overloading



Program 14.14: Overloading operator=: RECAP (Module 14)

Module M18

Partha Pratim
Das

Objectives &
Outlines

Operator
Function

Non-Member

Member

Rules

Global Function

public data
members

private data
members

Member Function

operator+

operator=

Unary Operators

Module Summary

```
#include <iostream>
#include <cstdlib>
#include <cstring>
using namespace std;

class String { public: char *str_; size_t len_;
    String(char *s) : str_(strdup(s)), len_(strlen(str_)) { }           // ctor
    String(const String& s) : str_(strdup(s.str_)), len_(s.len_) { }    // cctor
    ~String() { free(str_); }                                           // dtor
    String& operator=(const String& s) {
        if (this != &s) { free(str_); str_ = strdup(s.str_); len_ = s.len_; }
        return *this;
    }
    void print() { cout << "(" << str_ << ": " << len_ << ")" << endl; }
};

int main() { String s1 = "Football", s2 = "Cricket";
    s1.print(); s2.print();
    s1 = s1; s1.print();
}
```

(Football: 8)
(Cricket: 7)
(Football: 8)

- Check for self-copy (**this != &s**)
- In case of self-copy, do nothing



Notes on Overloading operator=: RECAP (Module 14)

Module M18

Partha Pratim
Das

Objectives &
Outlines

Operator
Function

Non-Member

Member

Rules

Global Function

public data
members

private data
members

Member Function

operator+

operator=

Unary Operators

Module Summary

- Overloaded operator= may choose between *Deep* and *Shallow Copy* for Pointer Members
 - *Deep copy* allocates new space for the contents and copies the pointed data
 - *Shallow copy* merely copies the pointer value – hence, the new copy and the original pointer continue to point to the same data
- If `operator=` is not overloaded by the user, compiler provides a free one.
- Free `operator=` can make only a shallow copy
- If the constructor uses `operator new`, `operator=` should be overloaded
- If there is a need to define a copy constructor then `operator=` must be overloaded and vice-versa



Program 18.04: Overloading Unary Operators

Module M18

Partha Pratim Das

Objectives & Outlines

Operator Function

Non-Member

Member

Rules

Global Function

public data members

private data members

Member Function

operator+

operator=

Unary Operators

Module Summary

```
#include <iostream>
using namespace std;

class MyClass { int data; public:
    MyClass(int d): data(d) { }

    MyClass& operator++() { // Pre-increment:
        ++data;           // Operate and return the operated object
        return *this;
    }
    MyClass operator++(int) { // Post-Increment:
        MyClass t(data);    // Return the (copy of) object; operate the object
        ++data;
        return t;
    }
    void disp() { cout << "Data = " << data << endl; }
};

int main() {
    MyClass obj1(8); obj1.disp();
    MyClass obj2 = obj1++; obj2.disp(); obj1.disp();

    obj2 = ++obj1;
    obj2.disp(); obj1.disp();
}
```

● Output

Data = 8

Data = 8

Data = 9

Data = 10

Data = 10

● The **pre-operator** should first perform the operation (increment / decrement / other) and then return the object. Hence its return type should be **MyClass&** and it should return ***this**;

● The **post-operator** should perform the operation (increment / decrement / other) after it returns the original value. Hence it should copy the original object in a temporary **MyClass t**; and then **return t**. Its return type should be **MyClass** - by value



Program 18.05: Overloading Unary Operators: Pre-increment & Post Increment

Module M18

Partha Pratim Das

Objectives & Outlines

Operator Function

Non-Member

Member

Rules

Global Function

public data members

private data members

Member Function

operator+

operator=

Unary Operators

Module Summary

```
#include <iostream>
using namespace std;

class MyClass { int data;
public:
    MyClass(int d) : data(d) { }

    MyClass& operator++() { // Pre-Operator
        data *= 2;
        return *this;
    }
    MyClass operator++(int) { // Post-Operator
        MyClass t(data);
        data /= 3;
        return t;
    }
    void disp() { cout << "Data = " << data << endl; }
};

int main() {
    MyClass obj1(12); obj1.disp();
    MyClass obj2 = obj1++; obj2.disp(); obj1.disp();

    obj2 = ++obj1;
    obj2.disp(); obj1.disp();
}
```

● Output
Data = 12
Data = 12
Data = 4
Data = 8
Data = 8

- The **pre-operator** and the **post-operator** need not merely increment / decrement
- They may be used for any other computation as this example shows
- However, it is a good design practice to keep close to the native semantics of the operator



Module Summary

Module M18

Partha Pratim
Das

Objectives &
Outlines

Operator
Function

Non-Member

Member

Rules

Global Function

`public` data
members

`private` data
members

Member Function

`operator+`

`operator=`

Unary Operators

Module Summary

- Introduced operator overloading for user-defined types
- Illustrated methods of overloading operators using global functions and member functions
- Outlined semantics for overloading binary and unary operators



Module M19

Partha Pratim
Das

Objectives &
Outlines

Issues in Operator
Overloading

operator+

operator==

operator<<,
operator>>

Guidelines

Module Summary

Programming in Modern C++

Module M19: Overloading Operator for User-Defined Types: Part 2

Partha Pratim Das

Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur

ppd@cse.iitkgp.ac.in

All url's in this module have been accessed in September, 2021 and found to be functional



Module Recap

Module M19

Partha Pratim Das

Objectives & Outlines

Issues in Operator Overloading

`operator+`

`operator==`

`operator<<`,
`operator>>`

Guidelines

Module Summary

- Introduced operator overloading for user-defined types
- Illustrated methods of overloading operators using global functions and member functions
- Outlined semantics for overloading binary and unary operators



Module Objectives

Module M19

Partha Pratim
Das

Objectives &
Outlines

Issues in Operator
Overloading

`operator+`

`operator==`

`operator<<`,
`operator>>`

Guidelines

Module Summary

- Understand how to overload operators for a user-defined type (class)
- Understand the aspects of overloading by friend function and its advantages



Module Outline

Module M19

Partha Pratim
Das

Objectives & Outlines

Issues in Operator
Overloading

`operator+`

`operator==`

`operator<<`,
`operator>>`

Guidelines

Module Summary

- 1 Issues in Operator Overloading
- 2 `operator+`
- 3 `operator==`
- 4 `operator<<`, `operator>>`
- 5 Guidelines for Operator Overloading
- 6 Module Summary



Operator Function for UDT: RECAP (Module 18)

Module M19

Partha Pratim Das

Objectives & Outlines

Issues in Operator Overloading

operator+

operator==

operator<<, operator>>

Guidelines

Module Summary

- Operator Function options:

- Global Function
- Member Function
- friend Function

- Binary Operator:

```
MyType a, b; // An enum, struct or class
MyType operator+(const MyType&, const MyType&); // Global
MyType operator+(const MyType&); // Member
friend MyType operator+(const MyType&, const MyType&); // Friend
```

- Unary Operator:

```
MyType operator++(const MyType&); // Global
MyType operator++(); // Member
friend MyType operator++(const MyType&); // Friend
```

- Examples:

Expression	Function	Remarks
a + b	operator+(a, b)	global / friend
++a	operator++(a)	global / friend
a + b	a.operator+(b)	member
++a	a.operator++()	member



Issues in Operator Overloading

Module M19

Partha Pratim
Das

Objectives &
Outlines

Issues in Operator
Overloading

`operator+`

`operator==`

`operator<<`,
`operator>>`

Guidelines

Module Summary

NPTTEL

Issues in Operator Overloading



Issue 1: Extending operator+

Module M19

Partha Pratim Das

Objectives & Outlines

Issues in Operator Overloading

operator+

operator==

operator<<,
operator>>

Guidelines

Module Summary

- Consider a `Complex` class. We have learnt how to overload `operator+` to add two `Complex` numbers:

```
Complex d1(2.5, 3.2), d2(1.6, 3.3), d3;
```

```
d3 = d1 + d2; // d3 = 4.1 +j 6.5
```

- Now we want to extend the operator so that a `Complex` number and a real number (no imaginary part) can be added together:

```
Complex d1(2.5, 3.2), d2(1.6, 3.3), d3;
```

```
d3 = d1 + 6.2; // d3 = 8.7 +j 3.2
```

```
d3 = 4.2 + d2; // d3 = 5.8 +j 3.3
```

- We show why global operator function is not good for this
- We show why member operator function cannot do this
- We show how `friend` function achieves this



Issue 2: Overloading IO Operators: `operator<<`, `operator>>`

Module M19

Partha Pratim
Das

Objectives &
Outlines

Issues in Operator
Overloading

`operator+`

`operator==`

`operator<<`,
`operator>>`

Guidelines

Module Summary

- Consider a `Complex` class. Suppose we want to overload the streaming operators for this class so that we can write the following code:

```
Complex d;
```

```
cin >> d;
```

```
cout << d;
```

- Let us note that these operators deal with stream types defined in `iostream`, `ostream`, and `istream`:
 - `cout` is an `ostream` object
 - `cin` is an `istream` object
- We show why global operator function is not good for this
- We show why member operator function cannot do this
- We show how `friend` function achieves this



Extending operator+

Module M19

Partha Pratim
Das

Objectives &
Outlines

Issues in Operator
Overloading

operator+

operator==

operator<<,
operator>>

Guidelines

Module Summary

Extending operator+



Program 19.01: Extending operator+ with Global Function

Module M19

Partha Pratim Das

Objectives & Outlines

Issues in Operator Overloading

operator+

operator==

operator<<, operator>>

Guidelines

Module Summary

```
#include <iostream>
using namespace std;
class Complex { public: double re, im;
    explicit Complex(double r = 0, double i = 0): re(r), im(i) { } // No implicit conversion is allowed
    void disp() { cout << re << " + j " << im << endl; }
};
Complex operator+(const Complex &a, const Complex &b) { // Overload 1
    return Complex(a.re + b.re, a.im + b.im);
}
Complex operator+(const Complex &a, double d) { // Overload 2
    Complex b(d); return a + b; // Create temporary object and use Overload 1
}
Complex operator+(double d, const Complex &b) { // Overload 3
    Complex a(d); return a + b; // Create temporary object and use Overload 1
}
int main() { Complex d1(2.5, 3.2), d2(1.6, 3.3), d3;
    d3 = d1 + d2; d3.disp(); // d3 = 4.1 + j 6.5. Overload 1
    d3 = d1 + 6.2; d3.disp(); // d3 = 8.7 + j 3.2. Overload 2
    d3 = 4.2 + d2; d3.disp(); // d3 = 5.8 + j 3.3. Overload 3
}
```

- Works fine with global functions - 3 separate overloading are provided
- A bad solution as it breaks the encapsulation – as discussed in Module 18
- Let us try to use member function
- **Note:** A simpler solution uses Overload 1 and implicit casting (for this we need to remove `explicit` before constructor). But that too breaks encapsulation. We discuss this when we take up cast operators



Program 19.02: Extending operator+ with Member Function

Module M19

Partha Pratim Das

Objectives & Outlines

Issues in Operator Overloading

operator+

operator==

operator<<, operator>>

Guidelines

Module Summary

```
#include <iostream>
using namespace std;
class Complex { double re, im;
public:
    explicit Complex(double r = 0, double i = 0) : re(r), im(i) { } // No implicit conversion is allowed
    void disp() { cout << re << " + j " << im << endl; }
    Complex operator+(const Complex &a) { // Overload 1
        return Complex(re + a.re, im + a.im);
    }
    Complex operator+(double d) { // Overload 2
        Complex b(d); // Create temporary object
        return *this + b; // Use Overload 1
    }
};
int main() { Complex d1(2.5, 3.2), d2(1.6, 3.3), d3;
    d3 = d1 + d2; d3.disp(); // d3 = 4.1 + j 6.5. Overload 1
    d3 = d1 + 6.2; d3.disp(); // d3 = 8.7 + j 3.2. Overload 2

    //d3 = 4.2 + d2; // Overload 3 is not possible - needs an object on left
    //d3.disp();
}
```

- Overload 1 and 2 works
- Overload 3 cannot be done because the left operand is double – not an object
- Let us try to use friend function
- **Note:** This solution too avoids the feature of cast operators



Operator Overloading using friend

Module M19

Partha Pratim Das

Objectives & Outlines

Issues in Operator Overloading

operator+

operator==

operator<<,
operator>>

Guidelines

Module Summary

- Using global function, accessing **private** data members inside operator function is gets difficult
- It increases writing overhead, makes code complicated, else violates encapsulation
- As we saw till now most operators can actually be overloaded either by global function or member function, *But If the left operand is not an object of the class type then it cannot be overloaded through member function*
- To handle such situation, we require **friend** function
 - **Example:** For two objects **d1** & **d2** of the same class, we cannot overload (**constant + d2**) using member function. However, using **friend** function we can overload (**d1 + d2**), (**d1 + constant**), or (**constant + d2**)
 - **Reason:** While computing (**d1 + d2**) with member function, **d1** calls the **operator+()** and **d2** is passed as an argument. Similarly in (**d1 + constant**), **d1** calls the **operator+()** and **constant** is passed as an argument. But while calling (**constant + d2**) a **constant** cannot call the member function

Similar analysis will also hold when **d1** & **d2** are objects of different classes and we cannot add the operator to the class of **d1**

- So operators like **<<**, **>>**, relational (**<**, **>**, **==**, **!=**, **<=**, **>=**) should be overloaded through **friend**



Program 19.03: Extending operator+ with friend Function

Module M19

Partha Pratim Das

Objectives & Outlines

Issues in Operator Overloading

operator+

operator==

operator<<, operator>>

Guidelines

Module Summary

```
#include <iostream>
using namespace std;
class Complex { double re, im; public:
    explicit Complex(double r = 0, double i = 0) : re(r), im(i) { } // No implicit conversion is allowed
    void disp() { cout << re << " + j " << im << endl; }
    friend Complex operator+(const Complex &a, const Complex &b) { // Overload 1
        return Complex(a.re + b.re, a.im + b.im);
    }
    friend Complex operator+(const Complex &a, double d) { // Overload 2
        Complex b(d); // Create temporary object
        return a + b; // Use Overload 1
    }
    friend Complex operator+(double d, const Complex &b) { // Overload 3
        Complex a(d); // Create temporary object
        return a + b; // Use Overload 1
    }
};

int main() { Complex d1(2.5, 3.2), d2(1.6, 3.3), d3;
    d3 = d1 + d2; d3.disp(); // d3 = 4.1 + j 6.5. Overload 1
    d3 = d1 + 6.2; d3.disp(); // d3 = 8.7 + j 3.2. Overload 2
    d3 = 4.2 + d2; d3.disp(); // d3 = 5.8 + j 3.3. Overload 3
}
```

- Works fine with friend functions - 3 separate overloading are provided and Preserves the encapsulation too
- **Note:** A simpler solution uses only Overload 1 and implicit casting (for this we need to remove `explicit` before constructor) will be discussed when we take up cast operators



Overloading Comparison Operators

Module M19

Partha Pratim
Das

Objectives &
Outlines

Issues in Operator
Overloading

`operator+`

`operator==`

`operator<<`,
`operator>>`

Guidelines

Module Summary

Overloading Comparison Operators



Program 19.04: Overloading operator== for strings with friend Function

Module M19

Partha Pratim
Das

Objectives &
Outlines

Issues in Operator
Overloading

operator+

operator==

operator<<,
operator>>

Guidelines

Module Summary

```
#include <iostream>
#include <string>
#include <cstdlib>
#include <cstring>
using namespace std;
class MyStr { const char *name_; public:
    explicit MyStr(const char *s) : name_(strdup(s)) { } ~MyStr() { free((void *)name_); }
    friend bool operator==(const MyStr& s1, const MyStr& s2) { return !strcmp(s1.name_, s2.name_); } // 1
    friend bool operator==(const MyStr& s1, const string& s2) { return !strcmp(s1.name_, s2.c_str()); } // 2
    friend bool operator==(const string& s1, const MyStr& s2) { return !strcmp(s1.c_str(), s2.name_); } // 3
};
int main() {
    MyStr mS1("red"), mS2("red"), mS3("blue"); string sS1("red"), sS2("red"), sS3("blue");
    if (mS1 == mS2) cout << "Match "; else cout << "Mismatch "; // MyStr, MyStr: Overload 1
    if (mS1 == mS3) cout << "Match "; else cout << "Mismatch "; // MyStr, MyStr: Overload 1
    if (mS1 == sS2) cout << "Match "; else cout << "Mismatch "; // MyStr, string: Overload 2
    if (mS1 == sS3) cout << "Match "; else cout << "Mismatch "; // MyStr, string: Overload 2
    if (sS1 == mS2) cout << "Match "; else cout << "Mismatch "; // string, MyStr: Overload 3
    if (sS1 == mS3) cout << "Match "; else cout << "Mismatch "; // string, MyStr: Overload 3
    if (sS1 == sS2) cout << "Match "; else cout << "Mismatch "; // string, string: C++ Lib
    if (sS1 == sS3) cout << "Match "; else cout << "Mismatch "; // string, string: C++ Lib
}
```

Output: Match Mismatch Match Mismatch Match Mismatch Match Mismatch

- **MyStr** is a user-defined string class while **string** is from C++ Standard Library. These are compared here by **operator==**.

We can extend this to include comparison with **char*** and for other comparison operators



Overloading IO Operators

Module M19

Partha Pratim
Das

Objectives &
Outlines

Issues in Operator
Overloading

`operator+`

`operator==`

`operator<<`,
`operator>>`

Guidelines

Module Summary

Overloading IO Operators



Overloading IO Operators: `operator<<`, `operator>>`

Module M19

Partha Pratim Das

Objectives & Outlines

Issues in Operator Overloading

`operator+`

`operator==`

`operator<<`,
`operator>>`

Guidelines

Module Summary

- Consider `operator<<` for `Complex` class. This operator should take an `ostream` object (stream to write to) and a `Complex` (object to write). Further it allows to chain the output. So for the following code

```
Complex d1, d2;
```

```
cout << d1 << d2; // (cout << d1) << d2;
```

the signature of `operator<<` may be one of:

```
// Global function
```

```
ostream& operator<< (ostream& os, const Complex &a);
```

```
// Member function in ostream
```

```
ostream& ostream::operator<< (const Complex &a);
```

```
// Member function in Complex
```

```
ostream& Complex::operator<< (ostream& os);
```

- Object to write is passed by constant reference
- Return by reference for `ostream` object is used so that chaining would work



Program 19.05: Overloading IO Operators with Global Function

Module M19

Partha Pratim Das

Objectives & Outlines

Issues in Operator Overloading

operator+

operator==

operator<<, operator>>

Guidelines

Module Summary

```
#include <iostream>
using namespace std;
class Complex {
public: double re, im;
    Complex(double r = 0, double i = 0): re(r), im(i) { }
};
ostream& operator<<(ostream& os, const Complex &a) {
    os << a.re << " +j " << a.im << endl;
    return os;
}
istream& operator>>(istream& is, Complex &a) {
    is >> a.re >> a.im;
    return is;
}
int main() {
    Complex d;

    cin >> d;

    cout << d;
}
```

- Works fine with global functions
- A bad solution as it breaks the encapsulation – as discussed in Module 18
- Let us try to use member function



Overloading IO Operators with Member Function

Module M19

Partha Pratim Das

Objectives & Outlines

Issues in Operator Overloading

operator+

operator==

operator<<, operator>>

Guidelines

Module Summary

- Case 1: `operator<<` is a member in `ostream` class:

```
ostream& ostream::operator<< (const Complex &a);
```

This is not possible as `ostream` is a class in C++ standard library and we are not allowed to edit it to include the above signature

- Case 2: `operator<<` is a member in `Complex` class:

```
ostream& Complex::operator<< (ostream& os);
```

In this case, the invocation of streaming will change to:

```
d << cout; // Left operand is the invoking object
```

This certainly spoils the natural syntax

- **IO operators cannot be overloaded by member functions**
- **Let us try to use friend function**



Program 19.06: Overloading IO Operators with friend Function

Module M19

Partha Pratim
Das

Objectives &
Outlines

Issues in Operator
Overloading

operator+

operator==

operator<<,
operator>>

Guidelines

Module Summary

```
#include <iostream>
using namespace std;
class Complex { double re, im;
public:
    Complex(double r = 0, double i = 0): re(r), im(i) { }
    friend ostream& operator<<(ostream& os, const Complex &a);
    friend istream& operator>>(istream& is, Complex &a);
};
friend ostream& operator<<(ostream& os, const Complex &a) {
    os << a.re << " +j " << a.im << endl;
    return os;
}
friend istream& operator>>(istream& is, Complex &a) {
    is >> a.re >> a.im;
    return is;
}
int main() { Complex d;

    cin >> d;

    cout << d;
}
```

- Works fine with friend functions



Guidelines for Operator Overloading

Module M19

Partha Pratim
Das

Objectives &
Outlines

Issues in Operator
Overloading

`operator+`

`operator==`

`operator<<`,
`operator>>`

Guidelines

Module Summary

NPTEL

Guidelines for Operator Overloading



Guidelines for Operator Overloading

Module M19

Partha Pratim Das

Objectives & Outlines

Issues in Operator Overloading

`operator+`

`operator==`

`operator<<`,
`operator>>`

Guidelines

Module Summary

- Use *global function* when encapsulation is not a concern. For example, using `struct String { char* str; }` to wrap a C-string and overload `operator+` to concatenate strings and build a String algebra
- Use *member function* when the left operand is necessarily an object of a class where the operator function is a member. Specifically `operator=`, `operator new`, `operator new[]`, `operator delete` etc. must be member functions
- Use *friend function*, otherwise for operators like `<<`, `>>`, relational (`<`, `>`, `==`, `!=`, `<=`, `>=`) should be overloaded through *friend*
- While overloading an operator, try to *preserve its natural semantics* for built-in types as much as possible. For example, `operator+` in a Set class should compute union and NOT intersection
- Usually stick to the *parameter passing* conventions (built-in types by value and UDT's by constant reference)
- Decide on the *return type* based on the natural semantics for built-in types as illustrated in the examples
- Consider the *effect of casting* on operands
- Only overload the operators that you may need (*minimal design*)



Module Summary

Module M19

Partha Pratim
Das

Objectives &
Outlines

Issues in Operator
Overloading

`operator+`

`operator==`

`operator<<`,
`operator>>`

Guidelines

Module Summary

- Several issues in operator overloading has been discussed
- Use of `friend` is illustrated in versatile forms of overloading with examples
- Discussed the overloading IO (streaming) operators
- Guidelines for operator overloading is summarized
- Use operator overloading to build algebra for:
 - Complex numbers
 - Fractions
 - Strings
 - Vector and Matrices
 - Sets
 - and so on ...



Module M20

Partha Pratim
Das

Objectives &
Outlines

namespace
Fundamental

namespace
Scenarios

namespace
Features

Nested namespace

using namespace

Global namespace

std namespace

namespaces are
Open

namespace
vis-a-vis class

Lexical Scope

Module Summary

Programming in Modern C++

Module M20: Namespace

Partha Pratim Das

Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur

ppd@cse.iitkgp.ac.in

All url's in this module have been accessed in September, 2021 and found to be functional



Module Recap

Module M20

Partha Pratim
Das

Objectives & Outlines

namespace
Fundamental

namespace
Scenarios

namespace
Features

Nested namespace

using namespace

Global namespace

std namespace

namespaces are
Open

namespace
vis-a-vis class

Lexical Scope

Module Summary

- Several issues in operator overloading has been discussed
- Use of `friend` is illustrated in versatile forms of overloading with examples
- Discussed the overloading IO (streaming) operators
- Guidelines for operator overloading is summarized
- Use operator overloading to build algebra for:
 - Complex numbers
 - Fractions
 - Strings
 - Vector and Matrices
 - Sets
 - and so on ...



Module Objectives

Module M20

Partha Pratim
Das

Objectives & Outlines

namespace
Fundamental

namespace
Scenarios

namespace
Features

Nested namespace

using namespace

Global namespace

std namespace

namespaces are
Open

namespace
vis-a-vis class

Lexical Scope

Module Summary

- Understand `namespace` as a free scoping mechanism to organize code better

NPTEL



Module Outline

Module M20

Partha Pratim
Das

Objectives & Outlines

namespace
Fundamental

namespace
Scenarios

namespace
Features

Nested namespace

using namespace

Global namespace

std namespace

namespaces are
Open

namespace
vis-a-vis class

Lexical Scope

Module Summary

- 1 namespace Fundamental
- 2 namespace Scenarios
- 3 namespace Features
 - Nested namespace
 - using namespace
 - Global namespace
 - std namespace
 - namespaces are Open
- 4 namespace vis-a-vis class
- 5 Lexical Scope
- 6 Module Summary



namespace Fundamental

Module M20

Partha Pratim
Das

Objectives &
Outlines

**namespace
Fundamental**

namespace
Scenarios

namespace
Features

Nested namespace

using namespace

Global namespace

std namespace

namespaces are
Open

namespace
vis-a-vis class

Lexical Scope

Module Summary

namespace **Fundamental**



namespace Fundamental

Module M20

Partha Pratim
Das

Objectives &
Outlines

namespace
Fundamental

namespace
Scenarios

namespace
Features

Nested namespace

using namespace

Global namespace

std namespace

namespaces are
Open

namespace
vis-a-vis class

Lexical Scope

Module Summary

- A namespace is a declarative region that provides a scope to the identifiers (the names of types, functions, variables, etc) inside it
- It is used to organize code into logical groups and to prevent name collisions that can occur especially when your code base includes multiple libraries
- namespace provides a class-like modularization without class-like semantics
- Oblivates the use of File Level Scoping of C (file) static



Program 20.01: namespace Fundamental

Module M20

Partha Pratim
Das

Objectives &
Outlines

namespace
Fundamental

namespace
Scenarios

namespace
Features

Nested namespace
using namespace

Global namespace
std namespace

namespaces are
Open

namespace
vis-a-vis class

Lexical Scope

Module Summary

● Example:

```
#include <iostream>
using namespace std;

namespace MyNameSpace {
    int myData; // Variable in namespace
    void myFunction() { cout << "MyNameSpace myFunction" << endl; } // Function in namespace
    class MyClass { int data; // Class in namespace
    public:
        MyClass(int d) : data(d) { }
        void display() { cout << "MyClass data = " << data << endl; }
    };
}

int main() {
    MyNameSpace::myData = 10; // Variable name qualified by namespace name
    cout << "MyNameSpace::myData = " << MyNameSpace::myData << endl;

    MyNameSpace::myFunction(); // Function name qualified by namespace name

    MyNameSpace::MyClass obj(25); // Class name qualified by namespace name
    obj.display();
}
```

- A name in a **namespace** is prefixed by the name of it
- Beyond scope resolution, all **namespace** items are treated as global



namespace Scenarios

Module M20

Partha Pratim
Das

Objectives &
Outlines

namespace
Fundamental

**namespace
Scenarios**

namespace
Features

Nested namespace

using namespace

Global namespace

std namespace

namespaces are
Open

namespace
vis-a-vis class

Lexical Scope

Module Summary

namespace **Scenarios**



Scenario 1: Redefining a Library Function

Program 20.02

- `cstdlib` has a function `int abs(int n);` that returns the absolute value of parameter `n`
- You need a special `int abs(int n);` function that returns the absolute value of parameter `n` if `n` is between -128 and 127. Otherwise, it returns 0
- **Once you add your `abs`, you cannot use the `abs` from library! It is hidden and gone!**
- **namespace comes to your rescue**

Name-hiding: `abs()`

```
#include <iostream>
#include <cstdlib>

int abs(int n) {
    if (n < -128) return 0;
    if (n > 127) return 0;
    if (n < 0) return -n;
    return n;
}

int main() { std::cout << abs(-203) << " "
                << abs(-6) << " "
                << abs(77) << " "
                << abs(179) << std::endl;
            }
// Output: 0 6 77 0
```

namespace: `abs()`

```
#include <iostream>
#include <cstdlib>
namespace myNS {
    int abs(int n) {
        if (n < -128) return 0;
        if (n > 127) return 0;
        if (n < 0) return -n;
        return n;
    }
}

int main() { std::cout << myNS::abs(-203) << " "
                << myNS::abs(-6) << " "
                << myNS::abs(77) << " "
                << myNS::abs(179) << std::endl;
            }
// Output: 0 6 77 0
std::cout << abs(-203) << " " << abs(-6) << " "
                << abs(77) << " " << abs(179) << std::endl;
// Output: 203 6 77 179
}
```



Scenario 2: Students' Record Application: The Setting

Program 20.03

Module M20

Partha Pratim
Das

Objectives &
Outlines

namespace
Fundamental

namespace
Scenarios

namespace
Features

Nested namespace
using namespace

Global namespace

std namespace
namespaces are
Open

namespace
vis-a-vis class

Lexical Scope

Module Summary

- An organization is developing an application to process students records
- `class St` for Students and `class StReg` for list of Students are:

```
#include <iostream>
#include <cstring>
using namespace std;
class St { public: // A Student
    typedef enum GENDER { male = 0, female };
    St(char *n, GENDER g) : name(strcpy(new char[strlen(n) + 1], n)), gender(g) { }
    void setRoll(int r) { roll = r; } // Set roll while adding the student
    GENDER getGender() { return gender; } // Get the gender for processing
    friend ostream& operator<< (ostream& os, const St& s) { // Print a record
        cout << ((s.gender == St::male) ? "Male " : "Female ")
            << s.name << " " << s.roll << endl;
        return os;
    }
private: char *name; GENDER gender; // name and gender provided for the student
        int roll; // roll is assigned by the system
};
class StReg { // Students' Register
    St **rec; /* List of students */ int nStudents; // Number of student
public: StReg(int size) : rec(new St*[size]), nStudents(0) { }
    void add(St* s) { rec[nStudents] = s; s->setRoll(++nStudents); }
    St *getStudent(int r) { return (r == nStudents + 1) ? 0 : rec[r - 1]; }
};
```

- The classes are included in a header file `Students.h`



Scenario 2: Students' Record Application: Team at Work

Program 20.03

- Two engineers – **Sabita** and **Niloy** – are assigned to develop processing applications for male and female students respectively. Both are given the **Students.h** file
- The lead **Purnima** of **Sabita** and **Niloy** has the responsibility to integrate what they produce and prepare a single application for both male and female students. The engineers produce:

Processing for males by **Sabita**

```
////////// App1.cpp //////////
#include <iostream>
using namespace std;
#include "Students.h"
extern StReg *reg;
void ProcSt() { cout << "MALE STUDENTS: " << endl;
    int r = 1; St *s;
    while (s = reg->getStudent(r++))
        if (s->getGender() == St::male) cout << *s;
    cout << endl << endl;
    return;
}
////////// Main.cpp //////////
#include <iostream>
using namespace std;
#include "Students.h"
StReg *reg = new StReg(1000);
int main()
{ St s("Ravi", St::male); reg->add(&s); ProcSt(); }
```

Processing for females by **Niloy**

```
////////// App2.cpp //////////
#include <iostream>
using namespace std;
#include "Students.h"
extern StReg *reg;
void ProcSt() { cout << "FEMALE STUDENTS: " << endl;
    int r = 1; St *s;
    while (s = reg->getStudent(r++))
        if (s->getGender() == St::female) cout << *s;
    cout << endl << endl;
    return;
}
////////// Main.cpp //////////
#include <iostream>
using namespace std;
#include "Students.h"
StReg *reg = new StReg(1000);
int main()
{ St s("Rhea", St::female); reg->add(&s); ProcSt(); }
```



Scenario 2: Students' Record Application: Integration Nightmare: Program 20.03

Module M20

Partha Pratim Das

Objectives & Outlines

namespace Fundamental

namespace Scenarios

namespace Features

Nested namespace

using namespace

Global namespace

std namespace

namespaces are Open

namespace vis-a-vis class

Lexical Scope

Module Summary

- To integrate, **Purnima** prepares the following `main()` in her `Main.cpp` where she intends to call the processing functions for males (as prepared by **Sabita**) and for females (as prepared by **Niloy**) one after the other:

```
#include <iostream>
using namespace std;
#include "Students.h"

void ProcSt(); // Function from App1.cpp by Sabita
void ProcSt(); // Function from App2.cpp by Niloy

StReg *reg = new StReg(1000);

int main() {
    St s1("Rhea", St::female); reg->add(&s1);
    St s2("Ravi", St::male); reg->add(&s2);

    ProcSt(); // Function from App1.cpp by Sabita
    ProcSt(); // Function from App2.cpp by Niloy
}
```

- **But the integration failed due to name clashes**
- **Both use the same signature `void ProcSt();` for their respective processing function. Actually, they have several functions, classes, and variables in their respective development with the same name and with same / different purposes**
- **How does **Purnima** perform the integration without major changes in the codes? – namespace**



Scenario 2: Students' Record Application: Wrap in namespace

Program 20.03

Module M20

Partha Pratim Das

Objectives & Outlines

namespace Fundamental

namespace Scenarios

namespace Features

Nested namespace

using namespace

Global namespace

std namespace

namespaces are Open

namespace vis-a-vis class

Lexical Scope

Module Summary

- Introduce two **namespaces** – **App1** for **Sabita** and **App2** for **Niloy**
- Wrap the respective codes:

Processing for males by **Sabita**

```
////////// App1.cpp //////////  
#include <iostream>  
using namespace std;  
#include "Students.h"  
  
extern StReg *reg;  
  
namespace App1 {  
    void ProcSt() {  
        cout << "MALE STUDENTS: " << endl;  
        int r = 1;  
        St *s;  
  
        while (s = reg->getStudent(r++))  
            if (s->getGender() == St::male)  
                cout << *s;  
  
        cout << endl << endl;  
        return;  
    }  
};
```

Programming in Modern C++

Processing for females by **Niloy**

```
////////// App2.cpp //////////  
#include <iostream>  
using namespace std;  
#include "Students.h"  
  
extern StReg *reg;  
  
namespace App2 {  
    void ProcSt() {  
        cout << "FEMALE STUDENTS: " << endl;  
        int r = 1;  
        St *s;  
  
        while (s = reg->getStudent(r++))  
            if (s->getGender() == St::female)  
                cout << *s;  
  
        cout << endl << endl;  
        return;  
    }  
};
```

Partha Pratim Das

M20.13



Scenario 2: Students' Record Application: A Good Night's Sleep Program 20.03

Module M20

Partha Pratim Das

Objectives & Outlines

namespace Fundamental

namespace Scenarios

namespace Features

Nested namespace

using namespace

Global namespace

std namespace

namespaces are Open

namespace vis-a-vis class

Lexical Scope

Module Summary

- Now the integration gets smooth:

```
using namespace std;
```

```
#include "Students.h"
```

```
namespace App1 { void ProcSt(); } // App1.cpp by Sabita
```

```
namespace App2 { void ProcSt(); } // App2.cpp by Niloy
```

```
StReg *reg = new StReg(1000);
```

```
int main() {  
    St s1("Ravi", St::female); reg->add(&s1);  
    St s2("Rhea", St::male); reg->add(&s2);
```

```
    App1::ProcSt(); // App1.cpp by Sabita
```

```
    App2::ProcSt(); // App2.cpp by Niloy
```

```
    return 0;
```

```
}
```

- Clashing names are made distinguishable by distinct names



namespace Features

Module M20

Partha Pratim
Das

Objectives &
Outlines

namespace
Fundamental

namespace
Scenarios

**namespace
Features**

Nested namespace
using namespace
Global namespace
std namespace
namespaces are
Open

namespace
vis-a-vis class

Lexical Scope

Module Summary

namespace Features



Program 20.04: Nested namespace

Module M20

Partha Pratim
Das

Objectives &
Outlines

namespace
Fundamental

namespace
Scenarios

namespace
Features

Nested namespace

using namespace

Global namespace

std namespace

namespaces are
Open

namespace
vis-a-vis class

Lexical Scope

Module Summary

- A namespace may be nested in another namespace

```
#include <iostream>
using namespace std;

int data = 0;           // Global name ::

namespace name1 {
    int data = 1;       // In namespace name1
    namespace name2 {
        int data = 2;   // In nested namespace name1::name2
    }
}

int main() {
    cout << data << endl;           // 0
    cout << name1::data << endl;    // 1
    cout << name1::name2::data << endl; // 2

    return 0;
}
```



Program 20.05: Using using namespace and using for shortcut

Module M20

Partha Pratim
Das

Objectives &
Outlines

namespace
Fundamental

namespace
Scenarios

namespace
Features

Nested namespace
using namespace

Global namespace
std namespace
namespaces are
Open

namespace
vis-a-vis class

Lexical Scope

Module Summary

- Using `using namespace` we can avoid lengthy prefixes

```
#include <iostream>
using namespace std;
```

```
namespace name1 {
    int v11 = 1;
    int v12 = 2;
}
namespace name2 {
    int v21 = 3;
    int v22 = 4;
}
```

```
using namespace name1; // All symbols of namespace name1 will be available
using name2::v21;      // Only v21 symbol of namespace name2 will be available
```

```
int main() {
    cout << v11 << endl;      // name1::v11
    cout << name1::v12 << endl; // name1::v12
    cout << v21 << endl;      // name2::v21
    cout << name2::v21 << endl; // name2::v21
    cout << v22 << endl;      // Treated as undefined
}
```



Program 20.06: Global namespace

Module M20

Partha Pratim
Das

Objectives &
Outlines

namespace
Fundamental

namespace
Scenarios

namespace
Features

Nested namespace

using namespace

Global namespace

std namespace

namespaces are
Open

namespace
vis-a-vis class

Lexical Scope

Module Summary

- `using` or `using namespace` hides some of the names

```
#include <iostream>
using namespace std;

int data = 0;           // Global Data

namespace name1 {
    int data = 1;       // namespace Data
}

int main() {
    using name1::data;

    cout << data << endl;           // 1 // name1::data -- Hides global data
    cout << name1::data << endl;    // 1
    cout << ::data << endl;         // 0 // ::data -- global data
}
```

- Items in Global namespace may be accessed by scope resolution operator (`::`)



Program 20.07: std Namespace

Module M20

Partha Pratim
Das

Objectives &
Outlines

namespace
Fundamental

namespace
Scenarios

namespace
Features

Nested namespace
using namespace
Global namespace

std namespace
namespaces are
Open

namespace
vis-a-vis class

Lexical Scope

Module Summary

- Entire C++ Standard Library is put in its own namespace, called std

Without using using std

```
#include <iostream>
```

```
int main() {  
    int num;  
    std::cout << "Enter a value: " ;  
    std::cin >> num;  
    std::cout << "value is: " ;  
    std::cout << num ;  
}
```

- Here, `cout`, `cin` are explicitly qualified by their `namespace`. So, to write to standard output, we specify `std::cout`; to read from standard input, we use `std::cin`

- It is useful if a few library is to be used; no need to add entire std library to the global `namespace`

With using using std

```
#include <iostream>  
using namespace std;
```

```
int main() {  
    int num;  
    cout << "Enter a value: " ;  
    cin >> num;  
    cout << "value is: " ;  
    cout << num ;  
}
```

- By the statement `using namespace std;` `std namespace` is brought into the current `namespace`, which gives us direct access to the names of the functions and classes defined within the library without having to qualify each one with `std::`
- When several libraries are to be used it is a convenient method



Program 20.08: namespaces are Open

Module M20

Partha Pratim
Das

Objectives &
Outlines

namespace
Fundamental

namespace
Scenarios

namespace
Features

Nested namespace

using namespace

Global namespace

std namespace

namespaces are
Open

namespace
vis-a-vis class

Lexical Scope

Module Summary

- namespace are open: New Declarations can be added

```
#include <iostream>
using namespace std;

namespace open // First definition
{ int x = 30; }

namespace open // Additions to the last definition
{ int y = 40; }

int main() {
    using namespace open; // Both x and y would be available

    x = y = 20;
    cout << x << " " << y ;
}
```

Output: 20 20



namespace vis-a-vis class

Module M20

Partha Pratim
Das

Objectives &
Outlines

namespace
Fundamental

namespace
Scenarios

namespace
Features

Nested namespace

using namespace

Global namespace

std namespace

namespaces are
Open

namespace
vis-a-vis class

Lexical Scope

Module Summary

namespace **vis-a-vis** class



namespace vis-a-vis class

Module M20

Partha Pratim
Das

Objectives &
Outlines

namespace
Fundamental

namespace
Scenarios

namespace
Features

Nested namespace
using namespace
Global namespace
std namespace
namespaces are
Open

namespace
vis-a-vis class

Lexical Scope

Module Summary

namespace

- Every namespace is not a class
- A namespace can be reopened and more declaration added to it
- No instance of a namespace can be created
- using-declarations can be used to short-cut namespace qualification
- A namespace may be unnamed

class

- Every class defines a namespace
- A class cannot be reopened
- A class has multiple instances
- No using-like declaration for a class
- An unnamed class is not allowed



Lexical Scope

Module M20

Partha Pratim
Das

Objectives &
Outlines

namespace
Fundamental

namespace
Scenarios

namespace
Features

Nested namespace

using namespace

Global namespace

std namespace

namespaces are
Open

namespace
vis-a-vis class

Lexical Scope

Module Summary

Lexical Scope



Lexical Scope

Module M20

Partha Pratim Das

Objectives & Outlines

namespace Fundamental

namespace Scenarios

namespace Features

Nested namespace

using namespace

Global namespace

std namespace

namespaces are Open

namespace vis-a-vis class

Lexical Scope

Module Summary

- The scope of a name binding – an association of a name to an entity, such as a variable – is the part of a computer program where the binding is valid: where the name can be used to refer to the entity
- C++ supports a variety of scopes:
 - **Expression Scope** – restricted to one expression, mostly used by compiler
 - **Block Scope** – create local context
 - **Function Scope** – create local context associated with a function
 - **Class Scope** – context for data members and member functions
 - **Namespace Scope** – grouping of symbols for code organization
 - **File Scope** – limit symbols to a single file
 - **Global Scope** – outer-most, singleton scope containing the whole program



Lexical Scope

Module M20

Partha Pratim
Das

Objectives &
Outlines

namespace
Fundamental

namespace
Scenarios

namespace
Features

Nested namespace
using namespace
Global namespace
std namespace
namespaces are
Open

namespace
vis-a-vis class

Lexical Scope

Module Summary

- Scopes may be named or Unnamed
 - Named Scope – Option to refer to the scope from outside
 - ▷ [Class Scope](#) – class name
 - ▷ [Namespace Scope](#) – namespace name or unnamed
 - ▷ [Global Scope](#) – "::"
 - Unnamed Scope
 - ▷ [Expression Scope](#)
 - ▷ [Block Scope](#)
 - ▷ [Function Scope](#)
 - ▷ [File Scope](#)
- Scopes may or may not be nested
 - Scopes that may be nested
 - ▷ [Block Scope](#)
 - ▷ [Class Scope](#)
 - ▷ [Namespace Scope](#)
 - Scopes that cannot be nested
 - ▷ [Expression Scope](#)
 - ▷ [Function Scope](#) – may contain [Class Scopes](#)
 - ▷ [File Scope](#) – will contain several other scopes
 - ▷ [Global Scope](#) – will contain several other scopes



Module Summary

Module M20

Partha Pratim
Das

Objectives &
Outlines

namespace
Fundamental

namespace
Scenarios

namespace
Features

Nested namespace

using namespace

Global namespace

std namespace

namespaces are
Open

namespace
vis-a-vis class

Lexical Scope

Module Summary

- Understood namespace as a scoping tool in c++
- Analyzed typical scenarios that namespace helps to address
- Studied several features of namespace
- Understood how namespace is placed in respect of different lexical scopes of C++



Tutorial T04

Partha Pratim
Das

Tutorial Recap

Objectives &
Outline

What is a
Library?

Static vs Shared

Our Library
Project

Static Library

Build Steps
Execution Trace

Shared Library

Build Steps
Execution Trace

Set Library Path
Dynamic Loading

Windows DLL

Tutorial Summary

Programming in Modern C++

Tutorial T04: How to build a C/C++ program?: Part 4: Static and Dynamic Library

Partha Pratim Das

Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur

ppd@cse.iitkgp.ac.in

All url's in this module have been accessed in September, 2021 and found to be functional



Tutorial Recap

Tutorial T04

Partha Pratim
Das

Tutorial Recap

Objectives &
Outline

What is a
Library?

Static vs Shared

Our Library
Project

Static Library

Build Steps

Execution Trace

Shared Library

Build Steps

Execution Trace

Set Library Path

Dynamic Loading

Windows DLL

Tutorial Summary

- Understood the build process and pipeline for C/C++ projects
- Learnt [make](#) for build automation

NPTEL



Tutorial Objective

Tutorial T04

Partha Pratim
Das

Tutorial Recap

Objectives &
Outline

What is a
Library?

Static vs Shared

Our Library
Project

Static Library

Build Steps

Execution Trace

Shared Library

Build Steps

Execution Trace

Set Library Path

Dynamic Loading

Windows DLL

Tutorial Summary

- To understand the role of libraries in C/C++ projects
- To learn about Static and Shared Libraries - how to build and use them

NPTEL



Tutorial Outline

Tutorial T04

Partha Pratim Das

Tutorial Recap

Objectives & Outline

What is a Library?

Static vs Shared

Our Library Project

Static Library

Build Steps

Execution Trace

Shared Library

Build Steps

Execution Trace

Set Library Path

Dynamic Loading

Windows DLL

Tutorial Summary

- 1 Tutorial Recap
- 2 What is a Library?
 - Static vs Shared
- 3 Our Library Project
- 4 Static Library Project
 - Build Steps
 - Execution Trace
- 5 Shared / Dynamic Library Project
 - Build Steps
 - Execution Trace
 - Set Library Path
 - Late Binding and Dynamic Loading
- 6 Dynamic Link Library (DLL) Project: Windows
- 7 Tutorial Summary



What is a Library?

Tutorial T04

Partha Pratim
Das

Tutorial Recap

Objectives &
Outline

What is a
Library?

Static vs Shared

Our Library
Project

Static Library

Build Steps

Execution Trace

Shared Library

Build Steps

Execution Trace

Set Library Path

Dynamic Loading

Windows DLL

Tutorial Summary

What is a Library?

Source: All Accessed 23-Sep-21

[Static library](#), Wikipedia

[Dynamic-link library](#), Wikipedia

[Library \(computing\)](#), Wikipedia

[C standard library](#), Wikipedia

[C++ Standard Library](#), Wikipedia



What is a Library?

Tutorial T04

Partha Pratim Das

Tutorial Recap

Objectives & Outline

What is a Library?

Static vs Shared

Our Library Project

Static Library

Build Steps

Execution Trace

Shared Library

Build Steps

Execution Trace

Set Library Path

Dynamic Loading

Windows DLL

Tutorial Summary

- A library is a package of code that is meant to be reused by many programs. Typically, a C / C++ library comes in two pieces:
 - A *header file* that defines the functionality the library is exposing (offering) to the programs using it. For example, `stdio.h`, `math.h`, etc. in C and `iostream`, `vector`, etc. in C++
 - A *pre-compiled binary* that contains the implementation of that functionality pre-compiled into machine language. For example, `glibc` is the **GNU C (Standard) Library** on Unix

Some libraries may be split into multiple files and/or have multiple header files

- Libraries are pre-compiled because
 - As libraries rarely change, they do not need to be recompiled often. They can just be reused in binary
 - As pre-compiled objects are in machine language, it prevents people from accessing or changing the source code protecting IP

Source: [A.1 — Static and dynamic libraries](#) (Accessed 23-Sep-21)



Types of Library

Tutorial T04

Partha Pratim Das

Tutorial Recap

Objectives & Outline

What is a Library?

Static vs Shared

Our Library Project

Static Library

Build Steps

Execution Trace

Shared Library

Build Steps

Execution Trace

Set Library Path

Dynamic Loading

Windows DLL

Tutorial Summary

- **Static Library**

- Consists of routines that are compiled and linked into the program. A program compiled with a static library would have the functionality of the library as a part of the executable
- Extensions:
 - ▷ **Unix:** `.a` (archive)
 - ▷ **Windows:** `.lib`

- **Dynamic / Shared Library**

- Consists of routines that are loaded into the application at run time
- Extensions:
 - ▷ **Unix:** `.so` (shared object)
 - ▷ **Windows:** `.dll` (dynamic link library)

- **Import Library**

- An import library automates the process of loading and using a dynamic library
- Extensions:
 - ▷ **Unix:** Shared object (`.so`) file doubles as both a dynamic and an import library
 - ▷ **Windows:** A small static library (`.lib`) of the same name as the dynamic library (`.dll`). The static library is linked into the program at compile time, and then the functionality of the dynamic library can effectively be used as if it were a static library



Static Library

Tutorial T04

Partha Pratim Das

Tutorial Recap

Objectives & Outline

What is a Library?

Static vs Shared

Our Library Project

Static Library

Build Steps

Execution Trace

Shared Library

Build Steps

Execution Trace

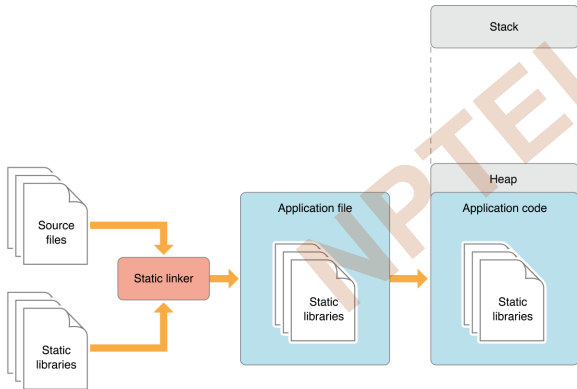
Set Library Path

Dynamic Loading

Windows DLL

Tutorial Summary

Static Library: Library Code is internal to Application



- Application *needs to recompile* - *difficult version management*
 - If library implementation changes - regular with version upgrade / bug fixes
 - And naturally, if library interface changes - infrequent
- *Large footprint* - especially *bad for mobile apps*
- *Multiple copies* of the same library may be loaded as part of different applications - *bad for mobile apps*
- *Fast in speed* as the library is already loaded and linked



Shared / Dynamic Library

Shared / Dynamic Library: Only Library reference is internal to Application - Library Code is external

Tutorial T04

Partha Pratim Das

Tutorial Recap

Objectives & Outline

What is a Library?

Static vs Shared

Our Library Project

Static Library

Build Steps

Execution Trace

Shared Library

Build Steps

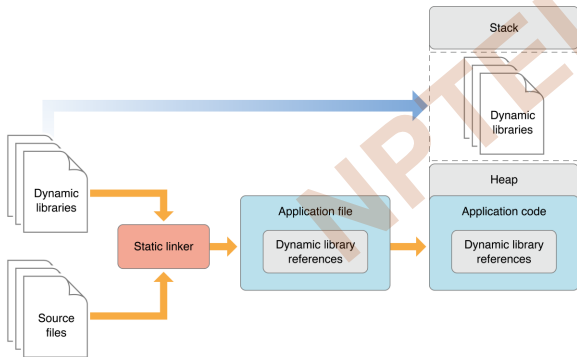
Execution Trace

Set Library Path

Dynamic Loading

Windows DLL

Tutorial Summary



- Application *does not need to recompile* - *easy version management*
 - If library implementation changes - regular with version upgrade / bug fixes
 - However, it will need to recompile (like the static library), if library interface changes - infrequent
- *Small footprint* - especially *good for mobile apps*
- *Single copy* of the library will be loaded for different applications - *good for mobile apps*
- The functions in the library needs to be *re-entrant*. Care is needed with static variables
- *Slow in speed* as the library may need to be loaded and linked at run-time



Static vs Shared Library

Tutorial T04

Partha Pratim Das

Tutorial Recap

Objectives & Outline

What is a Library?

Static vs Shared

Our Library Project

Static Library

Build Steps

Execution Trace

Shared Library

Build Steps

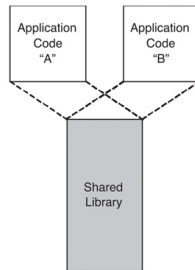
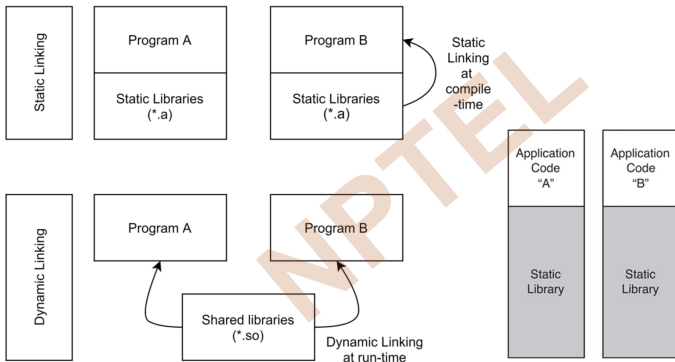
Execution Trace

Set Library Path

Dynamic Loading

Windows DLL

Tutorial Summary





Static vs Shared Library

Tutorial T04

Partha Pratim Das

Tutorial Recap

Objectives & Outline

What is a Library?

Static vs Shared

Our Library Project

Static Library

Build Steps

Execution Trace

Shared Library

Build Steps

Execution Trace

Set Library Path

Dynamic Loading

Windows DLL

Tutorial Summary

Property	Static Library	Shared Library
<i>Compilation</i>	Recompilation is required for changes in external files	No need to recompile the executable
<i>Linking time</i>	Happens as the last step of the compilation process	Are added during linking when executable file and libraries are added to the memory
<i>Import / Mechanism</i>	Are resolved in a caller at compile-time and copied into a target application by the linker	Get imported at the time of execution of target program by the OS
<i>Size</i>	Are bigger in size, because external programs are built in the executable file	Are smaller, because there is only one copy of shared library that is kept in memory
<i>External file changes</i>	Executable file will have to be recompiled if any changes were applied to external files	No need to recompile the executable - only the shared library is replaced
<i>Time / Performance</i>	Takes longer to execute, as loading into memory happens every time while executing	Faster because shared library is already in the memory
<i>Compatibility</i>	Never has compatibility issue, since all code is in one executable module	Programs are dependent on having a compatible library

Source: [Difference between Static and Shared libraries](#) and [Difference between Static and Shared libraries](#) (Accessed 23-Sep-21)



Our Library Project

Tutorial T04

Partha Pratim
Das

Tutorial Recap

Objectives &
Outline

What is a
Library?

Static vs Shared

Our Library
Project

Static Library

Build Steps

Execution Trace

Shared Library

Build Steps

Execution Trace

Set Library Path

Dynamic Loading

Windows DLL

Tutorial Summary

Our Library Project

Sources: All Accessed 26-Sep-21

Building And Using Static And Shared "C" Libraries

Shared libraries with GCC on Linux

MinGW Static and Dynamic Libraries

Static and Dynamic Libraries in C Language

Programming in Modern C++

Partha Pratim Das

T04.12



Our Library Project

Tutorial T04

Partha Pratim Das

Tutorial Recap

Objectives & Outline

What is a Library?

Static vs Shared

Our Library Project

Static Library

Build Steps

Execution Trace

Shared Library

Build Steps

Execution Trace

Set Library Path

Dynamic Loading

Windows DLL

Tutorial Summary

- We present a tiny project to illustrate the ideas of static and shared / dynamic libraries
- We use the same set of header and source files to create and use
 - Static Library
 - Shared / Dynamic Libraryand compare them
- First the projects are created with `gcc`. These can work on Unix as well as Windows (with `minGW`: `MinGW - Minimalist GNU for Windows`)
- Then we show Microsoft-specific process on Windows



Project Files

Tutorial T04

Partha Pratim
Das

Tutorial Recap

Objectives &
Outline

What is a
Library?

Static vs Shared

Our Library
Project

Static Library

Build Steps

Execution Trace

Shared Library

Build Steps

Execution Trace

Set Library Path

Dynamic Loading

Windows DLL

Tutorial Summary

```
/* lib_myMath.h: Header for my mathematical functions */ /* CPP guards omitted for brevity */
int myMax(int, int);
int myMin(int, int);
```

```
/* lib_myMath.c: Implementation for my mathematical functions */
#include "lib_myMath.h"
int myMax(int a, int b) { return a>b? a: b; }
int myMin(int a, int b) { return a<b? a: b; }
```

```
/* lib_myPrint.h: Header for my printing function */ /* CPP guards omitted for brevity */
void myPrint(const char*, int);
```

```
/* lib_myPrint.c: Implementation for my printing function */
#include <stdio.h>
#include "lib_myPrint.h"
void myPrint(const char *name, int a) { printf("%s: %d\n", name, a); }
```

```
/* myApp.c: My application */
#include <stdio.h>
#include "lib_myMath.h"
#include "lib_myPrint.h"
int main() {
    myPrint("Max(3, 5)", myMax(3, 5));
    myPrint("Min(3, 5)", myMin(3, 5));
}
```

Programming in Modern C++

Partha Pratim Das

T04.14



Project: Folders and Code Organization

Tutorial T04

Partha Pratim Das

Tutorial Recap

Objectives & Outline

What is a Library?

Static vs Shared

Our Library Project

Static Library

Build Steps

Execution Trace

Shared Library

Build Steps

Execution Trace

Set Library Path

Dynamic Loading

Windows DLL

Tutorial Summary

```
Home // Library demonstration project with library as well as application Home = Static or Shared
|
|---- app // Application files - will use library headers from ../inc and library from ../lib
|   |---- myApp.c      // Application Source file
|   |---- myApp.exe    // Application Executable
|   |---- myApp.o      // Application Object file
|   |---- myMakeApp.txt // Application Makefile
|
|---- inc // Headers to be included in application and library build
|   |---- lib_myMath.h
|   |---- lib_myPrint.h
|
|---- lib // Library files
|   |---- obj // Library object files
|       |---- lib_myMath.o
|       |---- lib_myPrint.o
|
|   |---- src // Library source files - will use library headers from ../../inc
|       |---- lib_myMath.c
|       |---- lib_myPrint.c
|
|   |---- lib_mylib.a      // Static Library binary file linked by the application
|   |---- lib_mylib.so     // Shared Library binary file linked by the application
|   |---- myMakeLibrary.txt // Library Makefile
```



Static Library Project

Tutorial T04

Partha Pratim
Das

Tutorial Recap

Objectives &
Outline

What is a
Library?

Static vs Shared

Our Library
Project

Static Library

Build Steps

Execution Trace

Shared Library

Build Steps

Execution Trace

Set Library Path

Dynamic Loading

Windows DLL

Tutorial Summary

Static Library Project

Sources: All Accessed 26-Sep-21

[A.1 — Static and dynamic libraries](#)

[A.2 — Using libraries with Visual Studio](#)

[A.3 — Using libraries with Code::Blocks](#)



Static Library Project: Build Steps

Tutorial T04

Partha Pratim Das

Tutorial Recap

Objectives & Outline

What is a Library?

Static vs Shared

Our Library Project

Static Library

Build Steps

Execution Trace

Shared Library

Build Steps

Execution Trace

Set Library Path

Dynamic Loading

Windows DLL

Tutorial Summary

- We can build this project by

```
$ gcc lib_myMath.c lib_myPrint.c myApp.c -o myApp
```
- Every time `myApp.c` is updated, we build `lib_myMath.c` and `lib_myPrint.c` even if there is no change. We can avoid the recompile by retaining the object files as:

```
$ gcc -c lib_myMath.c lib_myPrint.c
$ gcc lib_myMath.o lib_myPrint.o myApp.c -o myApp
```
- When we have many such files that rarely change, we would have a lot of such `.o` files to maintain. These can be bundled into an archive `lib_mylib.a` for ease of reference

```
$ ar rcs lib_mylib.a lib_myMath.o lib_myPrint.o
```

 - GNU `ar` utility creates, modifies, and extracts from *archives* (like ZIP) - holding a collection of multiple files in a structure that makes it possible to retrieve the individual files (called *members*)
 - Option `rcs` asks to create (`c`) an archive with replacement (`r`) of members and indexing (`s`)
 - For details check: [ar\(1\) — Linux manual page](#)
- Finally we use the `.a` file in place of the `.o`'s to link to `myApp.o`

```
$ gcc -o myApp myApp.c lib_myLib.a -L.
```

Alternately, we can place `lib_myLib.a` in default library path and link by `-l_mylib`

```
$ gcc -o myApp myApp.c -l_mylib -L.
```



Static Library Project: Makefiles

Tutorial T04

Partha Pratim Das

Tutorial Recap

Objectives & Outline

What is a Library?

Static vs Shared

Our Library Project

Static Library

Build Steps

Execution Trace

Shared Library

Build Steps

Execution Trace

Set Library Path

Dynamic Loading

Windows DLL

Tutorial Summary

Static Library makefile

```
# Variables
CC=gcc
AR=ar
SDIR=src
IDIR=./inc
ODIR=obj
CFLAGS=-I$(IDIR)
LFLAGS=-L.
AFLAGS=racs
# Macros
_DEPS = lib_myMath.h lib_myPrint.h
DEPS = $(patsubst %, $(IDIR)/%, $(DEPS))
_SRC = lib_myMath.c lib_myPrint.c
SRC = $(patsubst %, $(SDIR)/%, $(SRC))
_OBJ = lib_myMath.o lib_myPrint.o
OBJ = $(patsubst %, $(ODIR)/%, $(OBJ))
# Rules
$(ODIR)/%.o: $(SDIR)/%.c $(DEPS)
    $(CC) -c -o $@ $< $(CFLAGS) -I.
%.o: $(SDIR)/%.c $(DEPS)
    $(CC) -c -o $@ $< $(CFLAGS)
lib_mylib.a: $(OBJ)
    $(AR) $(AFLAGS) -o $@ $^
```

Application makefile

```
# Variables
CC=gcc
IDIR=inc
LDIR=lib
CFLAGS=-I../$(IDIR)
LFLAGS=-L../$(LDIR)
DEPS=
# Rules
%.o: %.c $(DEPS)
    $(CC) -c -o $@ $< $(CFLAGS)
myApp: myApp.o
    $(CC) -o myApp myApp.o -l_mylib $(LFLAGS)
```




Static Library Project: Execution Trace

Tutorial T04

Partha Pratim
Das

Tutorial Recap

Objectives &
Outline

What is a
Library?

Static vs Shared

Our Library
Project

Static Library

Build Steps

Execution Trace

Shared Library

Build Steps

Execution Trace

Set Library Path

Dynamic Loading

Windows DLL

Tutorial Summary

- Let us build and execute the project

```
// Build Library
```

```
D:\Library\Static\lib $ make -f myMakeLibrary.txt
```

```
gcc -c -o obj/lib_myMath.o src/lib_myMath.c -I../inc -I.
```

```
gcc -c -o obj/lib_myPrint.o src/lib_myPrint.c -I../inc -I.
```

```
ar rcs -o lib_mylib.a obj/lib_myMath.o obj/lib_myPrint.o
```

```
// Build Application
```

```
D:\Library\Static\app $ make -f myMakeApp.txt
```

```
gcc -c -o myApp.o myApp.c -I../inc
```

```
gcc -o myApp myApp.o -l_mylib -L../lib
```

```
// Run Application
```

```
D:\Library\Static\app $ myApp.exe
```

```
Max(3, 5): 5
```

```
Min(3, 5): 3
```

- So the static library is working as expected
- Next we check on the contents of various folders



Static Library Project: Directory Listing

Tutorial T04

Partha Pratim
Das

Tutorial Recap

Objectives &
Outline

What is a
Library?

Static vs Shared

Our Library
Project

Static Library

Build Steps

Execution Trace

Shared Library

Build Steps

Execution Trace

Set Library Path

Dynamic Loading

Windows DLL

Tutorial Summary

Directory of D:\Library\Static

26-09-2021	14:58	<DIR>	app
26-09-2021	14:58	<DIR>	inc
26-09-2021	14:58	<DIR>	lib

Directory of D:\Library\Static\app

24-09-2021	15:53	179	myApp.c
26-09-2021	11:35	42,348	myApp.exe
26-09-2021	11:35	954	myApp.o
25-09-2021	13:23	215	myMakeApp.txt

Directory of D:\Library\Static\inc

24-09-2021	13:17	66	lib_myMath.h
24-09-2021	13:17	54	lib_myPrint.h

Directory of D:\Library\Static\lib

26-09-2021	11:33	1,722	lib_mylib.a
25-09-2021	13:22	524	myMakeLibrary.txt

26-09-2021	14:58	<DIR>	obj
------------	-------	-------	-----

26-09-2021	14:58	<DIR>	src
------------	-------	-------	-----

Directory of D:\Library\Static\lib\obj

26-09-2021	11:33	716	lib_myMath.o
26-09-2021	11:33	778	lib_myPrint.o

Directory of D:\Library\Static\lib\src

24-09-2021	13:16	143	lib_myMath.c
24-09-2021	13:18	144	lib_myPrint.c



Shared / Dynamic Library Project

Tutorial T04

Partha Pratim
Das

Tutorial Recap

Objectives &
Outline

What is a
Library?

Static vs Shared

Our Library
Project

Static Library

Build Steps

Execution Trace

Shared Library

Build Steps

Execution Trace

Set Library Path

Dynamic Loading

Windows DLL

Tutorial Summary

Shared / Dynamic Library Project

Sources: All Accessed 26-Sep-21

A.1 — [Static and dynamic libraries](#)

A.2 — [Using libraries with Visual Studio](#)

A.3 — [Using libraries with Code::Blocks](#)



Shared Library Project: Build Steps

Tutorial T04

Partha Pratim Das

Tutorial Recap

Objectives & Outline

What is a Library?

Static vs Shared

Our Library Project

Static Library

Build Steps

Execution Trace

Shared Library

Build Steps

Execution Trace

Set Library Path

Dynamic Loading

Windows DLL

Tutorial Summary

- As in the static case, first we compile `lib_myMath.c` and `lib_myPrint.c` to create the object (`.o`) files using the option `-fPIC`:

```
$ gcc -fPIC -c lib_myMath.c lib_myPrint.c
```

- `-fPIC` stands to mean: Compile for *Position Independent Code* (PIC)
 - ▷ For a shared library, the binary of the library and the application are separate and will be separately loaded at run time
 - ▷ So when the object files are generated, we need that all jump calls and subroutine calls to use relative addresses, and not absolute addresses
 - ▷ `-fPIC` flag tells `gcc` to generate this type of code



Shared Library Project: Build Steps

Tutorial T04

Partha Pratim Das

Tutorial Recap

Objectives & Outline

What is a Library?

Static vs Shared

Our Library Project

Static Library

Build Steps

Execution Trace

Shared Library

Build Steps

Execution Trace

Set Library Path

Dynamic Loading

Windows DLL

Tutorial Summary

- Next step of building the library gets different now as we do not use `ar`. Rather we use `gcc` with the `-shared` option

```
$ gcc -shared -o lib_mylib.so lib_myMath.o lib_myPrint.o
```
- This creates a shared library `lib_mylib.so` where the extension `.so` stands for a *shared object*
- Finally we use the `.so` file to link to `myApp.o`

```
$ gcc -o myApp myApp.c lib_myLib.so -L.
```
- Alternately, we can place `lib_myLib.so` in default library path and link by `-l_mylib`

```
$ gcc -o myApp myApp.c -l_mylib
```



Shared Library Project: Makefiles

Tutorial T04

Partha Pratim Das

Tutorial Recap

Objectives & Outline

What is a Library?

Static vs Shared

Our Library Project

Static Library

Build Steps

Execution Trace

Shared Library

Build Steps

Execution Trace

Set Library Path

Dynamic Loading

Windows DLL

Tutorial Summary

Shared Library makefile

```
# Variables
CC=gcc
SDIR=src
IDIR=../inc
ODIR=obj
CFLAGS=-I$(IDIR)
LFLAGS=-L.

# Macros
_DEPS = lib_myMath.h lib_myPrint.h
DEPS = $(patsubst %, $(IDIR)/%, $(_DEPS))
_SRC = lib_myMath.c lib_myPrint.c
SRC = $(patsubst %, $(SDIR)/%, $(_SRC))
_OBJ = lib_myMath.o lib_myPrint.o
OBJ = $(patsubst %, $(ODIR)/%, $(_OBJ))

# Rules
$(ODIR)/%.o: $(SDIR)/%.c $(DEPS)
    $(CC) -fPIC -c -o $@ $< $(CFLAGS) -I.
lib_mylib.so: $(OBJ)
    $(CC) -shared -o $@ $^
```

Application makefile

```
# Variables
CC=gcc
IDIR=inc
LDIR=lib
CFLAGS=-I../$(IDIR)
LFLAGS=-L../$(LDIR)
DEPS=

# Rules
%.o: %.c $(DEPS)
    $(CC) -c -o $@ $< $(CFLAGS)

myApp: myApp.o
    $(CC) -o myApp myApp.o ../$(LDIR)/lib_mylib.so
```



Shared Library Project: Execution Trace

Tutorial T04

Partha Pratim Das

Tutorial Recap

Objectives & Outline

What is a Library?

Static vs Shared

Our Library Project

Static Library

Build Steps

Execution Trace

Shared Library

Build Steps

Execution Trace

Set Library Path

Dynamic Loading

Windows DLL

Tutorial Summary

- Let us build and execute the project

// Build Library

```
D:\Library\Shared\lib $ make -f myMakeLibrary.txt
gcc -fPIC -c -o obj/lib_myMath.o src/lib_myMath.c -I../inc -I.
gcc -fPIC -c -o obj/lib_myPrint.o src/lib_myPrint.c -I../inc -I.
gcc -shared -o lib_mylib.so obj/lib_myMath.o obj/lib_myPrint.o
```

// Build Application

```
D:\Library\Shared\app $ make -f myMakeApp.txt
gcc -c -o myApp.o myApp.c -I../inc
gcc -o myApp myApp.o ../lib/lib_mylib.so
```

// Run Application

```
D:\Library\Shared\app $ myApp.exe
```

Oops! The shared library is not found! The system does not know that lib_mylib.so is in

D:\Library\Shared\lib

myApp.exe - System Error



The code execution cannot proceed because lib_mylib.so was not found. Reinstalling the program may fix this problem.

OK



Shared Library Project: Execution Trace

Tutorial T04

Partha Pratim Das

Tutorial Recap

Objectives & Outline

What is a Library?

Static vs Shared

Our Library Project

Static Library

Build Steps

Execution Trace

Shared Library

Build Steps

Execution Trace

Set Library Path

Dynamic Loading

Windows DLL

Tutorial Summary

- If we copy `lib_mylib.so` to the application folder `D:\Library\Shared\app` (where `myApp.exe` resides), the problem goes away and the application runs successfully

```
// Run Application
```

```
D:\Library\Shared\app $ myApp.exe
```

```
Max(3, 5): 5
```

```
Min(3, 5): 3
```

- So the shared library is working as expected
 - However, copying the shared library to the application folder is not preferred as we would need to copy `lib_mylib.so` to every application that would use it.
 - We shall discuss a solution to this library path problem in the next section
- Next we check on the contents of various folders and compare the size of the libraries and applications in static and shared cases

Static Library		Shared Library		Remarks
<code>lib_mylib.a</code>	1,722	<code>lib_mylib.so</code>	27,749	File <code>.so</code> is larger than <code>.a</code> due to the overhead of exported references. With large number of functions in the library (as opposed to just 3) the relative overhead will go down
<code>myApp.exe</code>	42,348	<code>myApp.exe</code>	41,849	Shared <code>.exe</code> would be relatively much smaller with more functions in the library



Shared Library Project: Directory Listing

Tutorial T04

Partha Pratim Das

Tutorial Recap

Objectives & Outline

What is a Library?

Static vs Shared

Our Library Project

Static Library

Build Steps

Execution Trace

Shared Library

Build Steps

Execution Trace

Set Library Path

Dynamic Loading

Windows DLL

Tutorial Summary

Directory of D:\Library\Shared

26-09-2021	14:58	<DIR>	app
26-09-2021	14:58	<DIR>	inc
26-09-2021	14:58	<DIR>	lib

Directory of D:\Library\Shared\app

24-09-2021	15:53		179 myApp.c
26-09-2021	11:45		41,849 myApp.exe
26-09-2021	11:45		954 myApp.o
26-09-2021	09:41		220 myMakeApp.txt

Directory of D:\Library\Shared\inc

24-09-2021	13:17		66 lib_myMath.h
24-09-2021	13:17		54 lib_myPrint.h

Directory of D:\Library\Shared\lib

26-09-2021	11:44		27,749 lib_mylib.so
26-09-2021	10:38		457 myMakeLibrary.txt

26-09-2021	14:58	<DIR>	obj
------------	-------	-------	-----

26-09-2021	14:58	<DIR>	src
------------	-------	-------	-----

Directory of D:\Library\Shared\lib\obj

26-09-2021	11:44		716 lib_myMath.o
26-09-2021	11:44		778 lib_myPrint.o

Directory of D:\Library\Shared\lib\src

24-09-2021	13:16		143 lib_myMath.c
24-09-2021	13:18		144 lib_myPrint.c



Shared Library Project: Set Library Path

Tutorial T04

Partha Pratim Das

Tutorial Recap

Objectives & Outline

What is a Library?

Static vs Shared

Our Library Project

Static Library

Build Steps

Execution Trace

Shared Library

Build Steps

Execution Trace

Set Library Path

Dynamic Loading

Windows DLL

Tutorial Summary

- We managed to make the application work by copying the shared library to the application folder. However, this is not preferred as we would need to copy `lib_mylib.so` to every application that would use it
- So we need to tell the location for a shared library to the system. This can be done in one of three ways:
 - Place the shared library file in one of the *default library folders* like `/usr/local/lib`, `/usr/local/lib64`, `/usr/lib` and `/usr/lib64` (on Unix) or like `C:\Windows\system32` and `C:\Windows` (on Windows). Refer to the system manual details on these folders
 - Add the folder of the library to the library search folders by *setting path / environment variables*:
 - ▷ Windows: Use `PATH`
 - ▷ Unix: Use `LD_LIBRARY_PATH`
 - We may also *Dynamically Load / Unload* a library at the run-time. This is known as *late binding*. This is achieved by using `dlopen()`, `dlsym()`, and `dlclose()` from `dlfcn.h` header



Shared Library Project: Set Library Path: Windows

Tutorial T04

Partha Pratim Das

Tutorial Recap

Objectives & Outline

What is a Library?

Static vs Shared

Our Library Project

Static Library

Build Steps

Execution Trace

Shared Library

Build Steps

Execution Trace

Set Library Path

Dynamic Loading

Windows DLL

Tutorial Summary

- **Windows:** Set environment variables `PATH` to include the folder of the shared library

```
// Check PATH
```

```
D:\Library\Shared\app $ PATH
```

```
PATH=C:\Windows\system32;...
```

```
// Set PATH
```

```
D:\Library\Shared\app $ SET PATH=%PATH%;D:\Library\Shared\lib
```

```
// Check PATH
```

```
D:\Library\Shared\app $ PATH
```

```
PATH=C:\Windows\system32;...;D:\Library\Shared\app
```

```
// Run Application
```

```
D:\Library\Shared\app $ myApp.exe
```

```
Max(3, 5): 5
```

```
Min(3, 5): 3
```

- `PATH` would be reset when we end the command session. Need to set it every time
- We may set the folder in `PATH` Environment Variable - Use (preferred) or System to retain it across sessions. Refer: [How to set the path and environment variables in Windows](#)



Shared Library Project: Set Library Path: Unix

Tutorial T04

Partha Pratim
Das

Tutorial Recap

Objectives &
Outline

What is a
Library?

Static vs Shared

Our Library
Project

Static Library

Build Steps

Execution Trace

Shared Library

Build Steps

Execution Trace

Set Library Path

Dynamic Loading

Windows DLL

Tutorial Summary

- **Unix:** Set environment variable `LD_LIBRARY_PATH` to include the directory of the shared library as follows:

- For `tcsh` or `csh`:

```
$ setenv LD_LIBRARY_PATH /full/path/to/library/directory:${LD_LIBRARY_PATH}
```

- For `sh`, `bash` and similar:

```
LD_LIBRARY_PATH=/full/path/to/library/directory:${LD_LIBRARY_PATH}  
export LD_LIBRARY_PATH
```

Note:

- `LD_LIBRARY_PATH` is a list of directories in which to search for ELF libraries at execution time
- The items in the list are separated by either colons or semicolons, and there is no support for escaping either separator
- A zero-length directory name indicates the current working directory



Shared Library Project: Dynamic Loading

Tutorial T04

Partha Pratim Das

Tutorial Recap

Objectives & Outline

What is a Library?

Static vs Shared

Our Library Project

Static Library

Build Steps

Execution Trace

Shared Library

Build Steps

Execution Trace

Set Library Path

Dynamic Loading

Windows DLL

Tutorial Summary

- We can link shared libraries to a process anytime during its life without automatically loading the shared library by the dynamic loader
- We can do this by using the 'dl' library – load a shared library, reference any of its symbols, call any of its functions, and finally detach it from the process when no longer needed
- This is useful when there may be multiple shared libraries for the same (similar) purpose and we get to know which one to link only at the run-time
- The steps involved are:
 - Load the library from its path using `dlopen()` and get its *handle*
 - Use the *handle* to get access (*function pointers*) to the specific functions we intend to use by `dlsym()`
 - Use the *function pointers* to call the functions in the shared library
 - Finally, unload the library with `dlclose()`
- We first present a simple schematic example



Shared Library Project: Late Binding

- Known as *Late Binding* as the actual functions to be called are decided only at the run-time

Application	Library
<pre>#include <dlfcn.h> int main() { void* handle = dlopen("hello.so", RTLD_LAZY); // RTLD_LAZY: Relocations shall be performed at an // implementation-defined time, ranging from the // time of the dlopen() call until the first // reference to a given symbol occurs typedef void (*hello_t)(); hello_t myHello = 0; myHello = (hello_t) dlsym(handle, "hello"); myHello(); dlclose(handle); }</pre>	<pre>#include <iostream> using namespace std; extern "C" void hello() { cout << "hello" << endl; }</pre>

- Now we present our project in the context of late binding



Shared Library Project: Dynamic Loading

Tutorial T04

Partha Pratim Das

Tutorial Recap

Objectives & Outline

What is a Library?

Static vs Shared

Our Library Project

Static Library

Build Steps

Execution Trace

Shared Library

Build Steps

Execution Trace

Set Library Path

Dynamic Loading

Windows DLL

Tutorial Summary

```
/* myDynamicApp.c */
#include <stdio.h>
#include <stdlib.h>
#include "lib_myMath.h"
#include "lib_myPrint.h"
#include <dlfcn.h> /* defines dlopen(), dlsym(), and dlclose() etc. */
int main() { void* lib_handle; /* handle of the opened library */
    const char* error_msg; /* Pointer to an error string */
    /* Load the shared library */
    lib_handle = dlopen("D:\\Library\\Shared\\lib\\lib_mylib.so", RTLD_LAZY); /* Library path - dynamic */
    if (!lib_handle) { error_msg = dlerror(); goto Error; }
    /* Define function pointers */
    int (*myMax)(int, int); int (*myMin)(int, int); void (*myPrint)(const char*, int);
    /* Locate the functions in the library. Pick by name and assign to the function pointer */
    myMax = dlsym(lib_handle, "myMax"); if (error_msg = dlerror()) goto Error;
    myMin = dlsym(lib_handle, "myMin"); if (error_msg = dlerror()) goto Error;
    myPrint = dlsym(lib_handle, "myPrint"); if (error_msg = dlerror()) goto Error;
    /* Call the functions */
    (*myPrint)("Max(3, 5)", (*myMax)(3, 5));
    (*myPrint)("Min(3, 5)", (*myMin)(3, 5));
    /* Unload the shared library */
    dlclose(lib_handle);
    return 0; /* Success */
Error: fprintf(stderr, "%s\n", error_msg); exit(1); /* Exit on error */
}
```



Dynamic Link Library (DLL) Project: Windows

Tutorial T04

Partha Pratim
Das

Tutorial Recap

Objectives &
Outline

What is a
Library?

Static vs Shared

Our Library
Project

Static Library

Build Steps

Execution Trace

Shared Library

Build Steps

Execution Trace

Set Library Path

Dynamic Loading

Windows DLL

Tutorial Summary

Sources: All Accessed 27-Sep-21

[Microsoft Visual C++ Static and Dynamic Libraries](#)

[Walkthrough: Create and use a static library](#)

[Walkthrough: Create and use your own Dynamic Link Library \(C++\)](#)

[How to link DLLs to C++ Projects](#)

Programming in Modern C++

Partha Pratim Das

T04.34



Microsoft Visual C++ Static and Dynamic Libraries

Tutorial T04

Partha Pratim Das

Tutorial Recap

Objectives & Outline

What is a Library?

Static vs Shared

Our Library Project

Static Library

Build Steps

Execution Trace

Shared Library

Build Steps

Execution Trace

Set Library Path

Dynamic Loading

Windows DLL

Tutorial Summary

- While static and shared library of Unix (specifically **GNU**) is available on Windows through **minGW**, Windows provides Microsoft specific support through MSVC
 - **Static Library**: To work with a static library:
 - ▷ We need to create a static library and an application project (that refers to the library project) using the MSVS IDE
 - Check: [Walkthrough: Create and use a static library](#)
 - ▷ No change in the library or application codes is needed
 - **Dynamic Link Library (DLL)**: To work with a DLL:
 - ▷ We need to create a DLL and an application project (that refers to the DLL project) using the MSVS IDE
 - Check: [Walkthrough: Create and use your own Dynamic Link Library \(C++\)](#)
 - ▷ We need to change the library and / or application codes with `dllexport` and `dllimport` - the Microsoft-specific storage-class attributes for C and C++
 - Check: [dllexport, dllimport](#)
 - ▷ These can be specified by `__declspec()` to export and import functions, data, and objects to or from a DLL
 - ▷ We modify the library project file to illustrate



Library Project Files: Using dllexport

Tutorial T04

Partha Pratim Das

Tutorial Recap

Objectives &
Outline

What is a
Library?

Static vs Shared

Our Library
Project

Static Library

Build Steps
Execution Trace

Shared Library

Build Steps
Execution Trace
Set Library Path
Dynamic Loading

Windows DLL

Tutorial Summary

```
/* lib_myMath.h: Header for my mathematical functions */ /* File changed for export */
__declspec(dllexport) int myMax(int, int); // storage-class attribute __declspec(dllexport) used
__declspec(dllexport) int myMin(int, int); // to export function from DLL. This is MS-specific

/* lib_myMath.c: Implementation for my mathematical functions */ /* No change */
#include "lib_myMath.h"
int myMax(int a, int b) { return a>b? a: b; }
int myMin(int a, int b) { return a<b? a: b; }

/* lib_myPrint.h: Header for my printing function */ /* File changed for export */
__declspec(dllexport) void myPrint(const char*, int);

/* lib_myPrint.c: Implementation for my printing function */ /* No change */
#include <stdio.h>
#include "lib_myPrint.h"
void myPrint(const char *name, int a) { printf("%s: %d\n", name, a); }

/* myApp.c: My application */ /* No change */
#include <stdio.h>
#include "lib_myMath.h"
#include "lib_myPrint.h"
int main() {
    myPrint("Max(3, 5)", myMax(3, 5));
    myPrint("Min(3, 5)", myMin(3, 5));
}
```



Library Project Files: Using dllimport

Tutorial T04

Partha Pratim Das

Tutorial Recap

Objectives & Outline

What is a Library?

Static vs Shared

Our Library Project

Static Library

Build Steps

Execution Trace

Shared Library

Build Steps

Execution Trace

Set Library Path

Dynamic Loading

Windows DLL

Tutorial Summary

- In the context of the DLL headers,

```
/* lib_myMath.h */
__declspec(dllexport) int myMax(int, int);
__declspec(dllexport) int myMin(int, int);
```

```
/* lib_myPrint.h */
__declspec(dllexport) void myPrint(const char*, int);
```

the application project may include the DLL headers or can just directly import the symbols by `dllimport`

Including DLL headers	Using <code>dllimport</code>
<pre>/* myApp.c: My application */ #include <stdio.h> #include "lib_myMath.h" #include "lib_myPrint.h" int main() { myPrint("Max(3, 5)", myMax(3, 5)); myPrint("Min(3, 5)", myMin(3, 5)); }</pre>	<pre>/* myApp.c: My application */ #include <stdio.h> #include "lib_myMath.h" // storage-class attribute __declspec(dllimport) used // to import function from DLL. This is MS-specific __declspec(dllimport) void myPrint(const char*, int); int main() { myPrint("Max(3, 5)", myMax(3, 5)); myPrint("Min(3, 5)", myMin(3, 5)); }</pre>



Tutorial Summary

Tutorial T04

Partha Pratim Das

Tutorial Recap

Objectives & Outline

What is a Library?

Static vs Shared

Our Library Project

Static Library

Build Steps

Execution Trace

Shared Library

Build Steps

Execution Trace

Set Library Path

Dynamic Loading

Windows DLL

Tutorial Summary

- Understood the role of libraries in C/C++ projects in reuse
- Learn about Static and Shared Libraries in Unix and Windows - how to build and use them
- Learnt about DLLs